

InfiniMind: A Learning-Optimized Large-Scale Brain-Computer Interface

Yeongwoo Jang*
Seoul National University
Seoul, Republic of Korea
yeongwoo.jang@snu.ac.kr

Daye Jung*
Seoul National University
Seoul, Republic of Korea
daye.jung@snu.ac.kr

Seunghyun Song
Seoul National University
Seoul, Republic of Korea
seunghyun.song@snu.ac.kr

Hunjun Lee†
Hanyang University
Seoul, Republic of Korea
hunjunlee@hanyang.ac.kr

Jangwoo Kim
Seoul National University
Seoul, Republic of Korea
jangwoo@snu.ac.kr

Abstract

Brain-computer interfaces (BCIs) provide an interactive closed-loop connection between the brain and a computer. By employing signal processors implanted within the brain, BCIs are driving innovations across various fields in neuroscience and medicine. Recent studies highlight the need to integrate non-volatile memories (NVMs) into the implanted system for large-scale applications. At the same time, they emphasize the importance of continual learning within the system to address non-stationarities in the recorded signals.

This work is the first to address the performance and lifetime issues of deploying learning on NVM-assisted BCI systems. To reduce excessive write overhead associated with learning support, we propose four optimization schemes tailored for BCI workloads. First, update filtering minimizes unnecessary writes by leveraging the sparse and recurring nature of BCI signals. Second, delta buffering exploits temporal locality inherent in BCI signals to minimize NVM writes. Third, out-of-place flushing reduces write amplification by packing multiple sub-page updates into a single page write. Fourth, waveform compression decreases the volume of written data by exploiting the structural characteristics of neural signals. We implement these optimizations in a memory controller and integrate it into the state-of-the-art NVM-assisted BCI system, realizing an end-to-end learning-optimized system. Evaluation results show that our system improves performance and lifetime by 5.39× and 23.52×, respectively, on representative continual learning algorithms.

CCS Concepts

• **Hardware** → **Neural systems**; **External storage**; • **Computer systems organization** → *Heterogeneous (hybrid) systems*; • **Computing methodologies** → *Lifelong machine learning*.

*Both authors contributed equally to this research.

†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1261-6/25/06

<https://doi.org/10.1145/3695053.3731067>

Keywords

Brain-Computer Interface, Continual Learning, Non-Volatile Memory, Low-Power System, Hardware Accelerator

ACM Reference Format:

Yeongwoo Jang, Daye Jung, Seunghyun Song, Hunjun Lee, and Jangwoo Kim. 2025. InfiniMind: A Learning-Optimized Large-Scale Brain-Computer Interface. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3695053.3731067>

1 Introduction

Brain-computer interfaces (BCIs) establish a connection between the brain and a computer to decode intended behaviors or detect brain malfunctions. BCI systems consist of neural probes (e.g., electrodes) that record neural signals from the brain, and processors that filter and analyze the signals. Subsequently, the results are either transmitted to an external device (e.g., a robotic arm) [40, 54, 119] or relayed back to the probes to electrically stimulate the brain [85, 111, 130]. This interactive system enables BCIs to restore body functions, treat neurological disorders, and advance neuroscience [44, 59, 90].

Among the various types of BCI systems, we focus on implantable BCIs integrated with processors (i.e., implantable BCI systems).

Implantable BCI. Implantable BCIs directly record and stimulate the brain through neural probes implanted beneath the skull. They provide precise brain-computer interfacing with high fidelity and spatiotemporal resolution [42, 131]. Prior studies utilize them to advance our understanding of the brain and to develop treatments for neurological disorders [90, 117]. For example, several clinically approved medical devices use implantable BCIs to detect and prevent seizures and brain degeneration [83, 105].

Processor-Integrated BCI. Emerging BCI systems integrate BCIs with processors to enable efficient and secure signal processing [17, 53, 59, 108, 109, 134]. This integration eliminates the need to transfer signals to external processors across the skull. As a result, it offers a promising approach for implantable BCIs, which otherwise suffer from high power consumption and latency due to the wireless communication overhead [53]. Additionally, its fully untethered and autonomous operation enhances system security [7].

Implantable BCI systems rely on specialized hardware accelerators to achieve practical and safe processing capabilities. These systems must meet sub-second latency requirements to support

medical applications, while operating within a power budget of tens of milliwatts to prevent overheating at the implant site [101, 130]. To satisfy the tight latency and power constraints, prior works design specialized compute units tightly coupled with on-chip SRAMs. As a representative example, HALO [53] combines multiple accelerators with a reconfigurable network-on-chip (NoC) to flexibly support various BCI tasks (e.g., seizure prediction, movement decoding).

Recent studies integrate non-volatile memories (NVMs) into BCI systems to support large-scale BCI workloads [93, 108, 113–115]. Implantable BCIs are rapidly scaling to record signals from an increasing number of neurons [13, 94, 96, 130]. At the same time, practical applications require complex algorithms, such as deep neural networks [31, 32, 106] and spike sorting [47, 56, 132], that process substantial amounts of data. To efficiently manage such large-scale workloads, recent works augment accelerator-based systems with NVMs, which offer lower power consumption and smaller area footprints compared to SRAMs [113]. For example, SCALO [108] extends HALO by utilizing NAND Flash to store neural recording histories and parameters (e.g., weight matrices) for complex, large-scale BCI workloads.

Despite their benefits, NVM-assisted BCI systems potentially violate the latency and power budgets when supporting learning in BCI applications. Learning emerges as an important problem in long-term clinical deployments of BCI systems [4, 11, 39, 60]. It is widely used to withstand variabilities in recorded signals due to drift and failure of neural probes, physiological changes in the brain, and fluctuations in neural activity [35, 49, 71, 96, 132]. In particular, on-device learning is essential for maintaining robust accuracy despite sudden and frequent changes, with real-time continuous adaptation to non-stationary signals. It also enhances security and stability by eliminating data transmission to external devices [20, 69, 126].

However, learning algorithms are inherently write-intensive, posing significant challenges for NVM-assisted systems. In NVM, writes incur substantially higher latency and energy consumption compared to reads. For instance, they incur up to $10\times$ higher latency in commercial NAND Flash products [80]. Moreover, as write operations wear out NVM cells, they reduce device lifetime and demand extra costs for wear-leveling mechanisms [87, 103, 128]. Figure 1 highlights the performance and lifetime limitations of a learning-enabled NVM-assisted BCI system. For evaluation, we reconfigure SCALO for online clustering, a representative BCI learning workload. The neural probe records signals from 1,024 channels at a 20 KHz sampling rate [130], processed by a multichannel spike sorting algorithm [86]. Figure 1a shows a $7.92\times$ performance degradation with learning, due to excessive NVM write overhead. Also, Figure 1b illustrates that NVM cells wear out within two months, requiring frequent surgical replacement of the implanted device.

To reduce excessive write operations, we propose four optimization schemes based on our analysis on BCI learning workloads.

- **Update Filtering:** We leverage the sparse and recurring nature of BCI signals to identify and filter out ineffective parameter changes during learning. Our method filters out up to 95% of the parameter updates with less than 0.55% accuracy drop.
- **Delta Buffering:** We exploit the temporal locality of BCI signals by buffering frequently updated parameters on a small on-chip

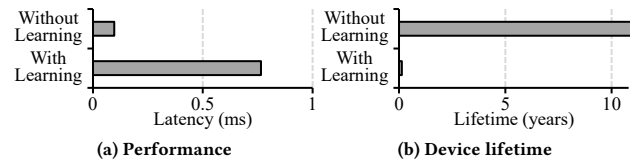


Figure 1: Limitations of an NVM-assisted BCI system when supporting a BCI learning workload.

buffer. The buffer effectively minimizes NVM writes by reusing up to 71% of the buffered data.

- **Out-of-Place Flushing:** We implement an out-of-place buffer flushing mechanism, inspired by a log-structured file system [95], to minimize write amplification. By packing multiple sub-page writes, it reduces the write amplification factor by up to 78%.
- **Waveform Compression:** We further minimize the write overhead by compressing the waveform data. Our waveform compression method operates on local segments, supporting fine-grained and irregular parameter updates. The compression reduces the write overhead by $4.50\times$ with less than 0.1% accuracy drop.

In this paper, we present InfiniMind, a learning-optimized BCI system designed for *lifelong* deployments at *immense* scale. To build InfiniMind, we incorporate our optimization schemes into SCALO, the state-of-the-art NVM-assisted BCI system. First, we develop a learning-optimized memory controller equipped with the four optimization schemes and a hardware-based lightweight flash translation layer (FTL). Next, we redesign the processing pipeline to support learning-specific computation patterns. Finally, we implement a dynamic NoC to manage variable-latency data movement and maximize processing pipeline utilization.

For evaluation, we deploy four representative BCI learning workloads to demonstrate the effectiveness of InfiniMind. We assess the speedup and lifetime improvements achieved by our optimization schemes when operating under the power budget. The results show that our optimizations effectively reduce write overhead, improving the average performance and lifetime by $5.39\times$ and $23.52\times$, respectively. As a result, InfiniMind achieves real-time processing capabilities and extends device lifetime, enabling practical and long-term deployments of BCI learning workloads.

In summary, we make the following contributions:

- **Important Problem Identification:** While designing learning-enabled systems is a critical challenge in the context of BCIs, it remains relatively unexplored compared to inference systems. This work is one of the first to identify and address the architectural challenges of supporting learning in modern BCI applications.
- **Learning-Optimized Memory Controller:** We propose a memory controller with four architectural solutions that leverage the unique characteristics of BCI signals and workloads. Our solutions effectively minimize intensive NVM write overhead in NVM-assisted BCI learning systems.
- **End-to-End BCI Processing System:** We seamlessly integrate our optimization schemes into SCALO, the state-of-the-art BCI inference system. The resulting design provides a highly efficient end-to-end BCI system that incorporates both learning and inference within the signal processing pipeline.

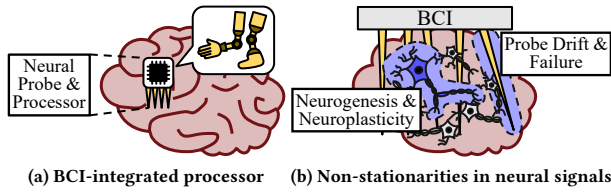


Figure 2: Implantable BCI system.

2 Background & Motivation

2.1 Implantable BCI System

Implantable BCI systems utilize neural probes placed on brain tissue to directly interface with the brain (Figure 2a). These neural probes capture electrical activities by detecting voltage fluctuations in nearby neurons [14]. The recorded signals are subsequently relayed to a dedicated processor integrated with the neural probes. It analyzes and decodes information from the recording to support various applications, such as movement decoding, seizure detection, and neuroscientific research [16, 19, 25, 44, 53, 54, 108, 125]. Based on the outcomes, the BCI system injects stimuli to treat disorders (e.g., seizure prevention [37, 59, 85, 111], prosthetic vision [84]).

An implantable BCI system must satisfy several key objectives.

① **Thermal and Power Constraints.** The implant should not heat the brain tissue by more than 1°C [64, 124, 136]. Prior studies show that consuming over 45 mW at the implant site potentially overheats and damages brain tissue [101, 130].

② **Latency Constraints.** The processor should operate within strict latency budgets. It should meet millisecond-scale latency requirements for real-time medical applications, such as 50 ms for movement decoding and 10 ms for seizure detection [108].

③ **Workload Coverage.** The processor should achieve high workload coverage. Specifically, BCIs are scaling up at a fast rate to record more neurons simultaneously, and the workloads are becoming more complex with larger amounts of application data [13, 94, 123].

④ **Device Lifetime.** BCI systems are implanted through burr holes in the skull, which poses various surgical risks (e.g., infections and tissue injuries [65, 68]). Therefore, to minimize surgical replacements of the implanted device, the system should operate reliably over a long period of time. For example, a commercial implantable BCI system guarantees a lifetime of at least 10 years [83].

2.2 Learning in BCI System

Researchers observe various forms of non-stationarity in recorded neural signals during long-term deployments of practical BCI systems (Figure 2b). First, high-density neural probes often experience drift over time, and individual recording channels are prone to failure after insertion [4, 39, 96]. For example, Neuralink [82] reports that 85% of channels retract within 100 days of implantation [122]. Second, the brain undergoes physical changes due to neurogenesis and neuroplasticity, which can change signal characteristics [38, 60]. For instance, clinical use cases demonstrate daily variations in neuronal firing rates and response sensitivities [78]. These non-stationarities gradually degrade the decoding accuracy over time [4, 71, 120].

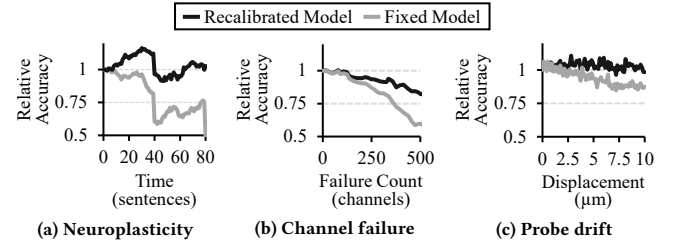


Figure 3: Relative accuracy of the recalibrated and fixed models for three different workloads with non-stationarities.

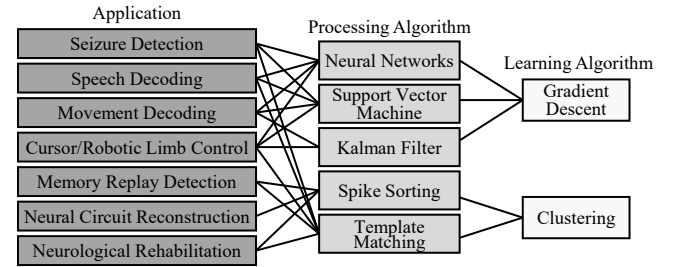


Figure 4: Categories of learning algorithms in BCI.

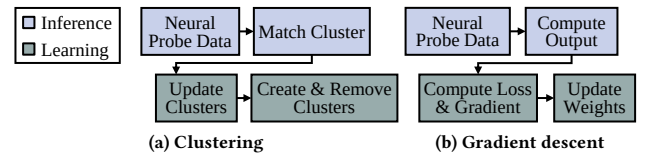


Figure 5: Representative BCI learning algorithms.

To mitigate the accuracy drop, BCI systems should frequently recalibrate the workload parameters through continual learning [35, 76, 120]. In Figure 3, we examine how different sources of non-stationarities affect accuracy in the BCI system: (1) neurogenesis and neuroplasticity, by tracking handwriting task accuracy over time (Figure 3a); (2) channel failure, by simulating the loss of 500 out of 1,024 channels in a reaching task (Figure 3b); and (3) probe drift, by simulating a $10\text{ }\mu\text{m}$ amplitude drift in a spike sorting workload (Figure 3c). The results show that fixed models without learning support experience an accuracy drop exceeding 15.3% across all scenarios. In contrast, recalibration effectively recovers most of the accuracy drop and even improves accuracy in some cases.

In this work, we focus on on-device learning, which enables autonomous, untethered operation without external connections. This eliminates the need for wireless data transmission, thereby enhancing security and stability [20, 69, 126] while also reducing power consumption [15]. Also, it effectively addresses sudden and frequent changes in recorded signals, ensuring robust accuracy.

We conduct an extensive analysis of various BCI processing algorithms and classify the continual learning algorithms into two representative categories (Figure 4).

Clustering (Figure 5a). Clustering [127] is an unsupervised learning algorithm for training various BCI algorithms such as spike

sorting [41, 66, 86, 98, 102] and template matching [1, 28, 45, 57, 73]. Each cluster represents a group of similar waveforms, and has a centroid waveform that represents the average of the cluster. First, the clustering algorithm calculates the similarity between the input waveform and the centroids of each cluster to identify the best-matching cluster (inference). If the best similarity exceeds a certain threshold, the input is assigned to the best-matching cluster, and the centroid is updated. Otherwise, the input waveform is used to create a new cluster. Also, clusters are periodically removed if they are not matched for a certain period (learning).

Gradient Descent (Figure 5b). Gradient descent [97] is an optimization algorithm used to find the local minimum of a loss function. It is widely employed to train machine learning models, such as neural networks [49, 61, 74, 129]. In a BCI system, the process begins by predicting an output from recorded neural signals using matrix multiplications and activation functions (inference). After completing a decoding trial (e.g., decoding a movement or a word), the system computes a loss function to quantify the prediction error. The loss function is calculated using either user-provided feedback or task-specific algorithms that generate pseudo-labels [35, 49]. Finally, the system computes the gradients based on the loss function and update the model parameters (learning).

2.3 Memory System for BCI Processor

2.3.1 SRAM-Only Systems. Prior studies design accelerators that tightly couple compute units with on-chip memory to meet the latency and power constraints of implantable BCI systems [53, 108]. They realize an efficient BCI system by specializing compute units for BCI applications. Each unit incorporates a large on-chip memory to store all the parameters required for target workloads. For example, HALO [53] integrates multiple processing elements (PEs), which are heterogeneous accelerators optimized for representative computational kernels. Each PE utilizes an on-chip memory to store workload parameters for 96 recording channels. By reconfiguring the NoC between PEs, HALO flexibly supports different application pipelines. While SRAM-only systems enable fast memory access, they suffer from high power consumption, limiting their scalability. For instance, in our analysis with OpenRAM [43], 1024-channel spike sorting on an SRAM-only system consumes 654.29 mW.

2.3.2 NVM-Assisted Systems. For broader workload coverage, recent BCI systems integrate NVMs into their memory systems [93, 108, 113–115]. NVMs provide low-power, high-density storage, making them a promising solution for handling the growing workload sizes in BCI applications. Recent BCI technologies enable the recording of signals from a greater number of neurons [6, 94, 130]. State-of-the-art BCIs, such as Neuralink [82] and NeuroPixels [34], are capable of recording signals from thousands of neurons. Moreover, modern BCI applications demand increasingly complex computational kernels with a growing number of parameters [35, 120]. To manage the scaling demands, SCALO [108] extends HALO by integrating NVMs to swap out parameters that exceed the on-chip memory capacity. Among the various NVM types, SCALO adopts NAND Flash for its high density and technological maturity.

NVM-assisted BCI systems have following characteristics.

Inefficient Write. A write operation in NVM is significantly slower than a read operation, as it requires changing the cell states using

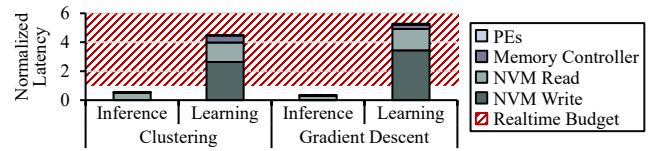


Figure 6: Latency breakdown of BCI learning workloads on an NVM-assisted system.

high-voltage pulses and performing repeated program-verify steps to reliably write the data. According to industrial reports, writes in commercial NAND Flash are typically 8–10× slower than reads [80]. **Page-Level Access.** NVMs access data at the page level, with the page sizes typically ranging from 2 KB to 16 KB [81]. The page-level access results in write amplification, as the NVM should program an entire page even if only a portion of the data is modified [46].

Lifetime. NVMs require block-level erase operations before writing new data to their pages. However, each block supports only a finite number of program/erase (P/E) cycles before wearing out, limiting device lifetime. For example, a commercial SLC NAND Flash endures up to 100,000 P/E cycles [80]. Therefore, NVMs adopt wear leveling techniques to distribute writes evenly across blocks.

2.3.3 Alternative Systems. Many memory systems utilize DRAM for its fast access and large storage capacity. However, its volatility makes it unsuitable as a replacement for NVM in BCI systems. Additionally, incorporating DRAM into the memory hierarchy increases power consumption, making it impractical for implantable BCIs. In our experiment with SimpleSSD [52], DRAM alone consumes 46.08 mW when used for caching parameters and mapping tables.

2.4 Learning on NVM-Assisted BCI System

Despite their high coverage, existing large-scale BCI systems with NVMs are unsuitable for BCI learning workloads. Learning algorithms are inherently write-intensive, causing significant overhead while shortening the device's lifetime. Clustering incurs fine-grained parameter updates whenever an input waveform matches a specific cluster. Although each update is typically small (e.g., 1.4 KB), the fine-grained update incurs write amplification and additional garbage collection overhead [75]. Gradient descent periodically updates the entire weights, incurring excessive NVM writes [22].

To examine the performance degradation, we profile clustering and gradient descent workloads on an NVM-assisted BCI system (Figure 6). We configure the system to operate within the power constraints of an implantable system and normalize latency against the real-time requirement of each workload. Refer to Section 5.1 for detailed setup. The results show that learning increases the normalized latency by 10.85× on average, causing the system to exceed its latency budget. This increase is due to slow NVM write operations and controller overhead for supporting garbage collection, which together account for more than 69.2% of the total latency.

We also analyze how BCI learning workloads affect NVM lifetime. Inference-only systems pose minimal endurance concerns, as their infrequent writes allow lifetimes exceeding 10 years. However, learning significantly increases write frequency, reducing NVM lifetime to 0.13 years for clustering and 0.4 years for gradient descent, necessitating periodic surgical replacements in practical scenarios.

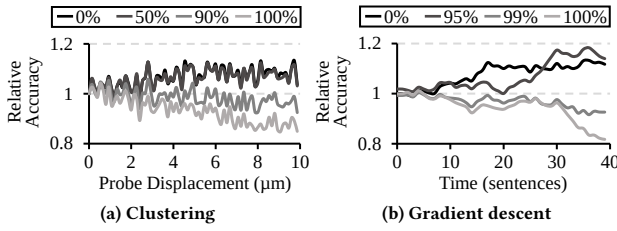


Figure 7: Relative accuracy with different filtering ratios.

3 Key Observations & Design Goals

To mitigate excessive write overhead, we conduct an extensive analysis of BCI learning workloads. Our analysis reveals four unique characteristics of BCI signals that can potentially mitigate the substantial NVM write overhead during learning. Accordingly, we set four design goals to fully exploit the identified characteristics for a learning-optimized NVM-assisted BCI system.

3.1 Redundant Updates in BCI Learning

In BCI systems, learning incurs a high degree of redundant parameter updates due to the recurrent and sparse nature of BCI signals. Our analysis shows that only a small fraction of updates contribute to the meaningful recalibration of model parameters. Therefore, most parameter updates can be filtered out without compromising learning accuracy. We analyze the two representative BCI learning algorithms (i.e., clustering and gradient descent) to quantify the redundancies. Each algorithm adopts a different filtering strategy to identify and eliminate redundant parameter updates.

Recurrence. Clustering leverages the recurring nature of BCI signals to filter out redundant parameter updates. Our analysis indicates that BCIs often record similar waveforms as some of the neurons are highly active over short periods. In some cases, this causes cluster updates to saturate quickly before the system encounters non-stationarities, rendering subsequent updates ineffective. Therefore, we propose to filter out updates when the similarity between cluster and recorded waveform exceeds a predefined threshold.

To assess the potential impact of filtering out updates, we profile the accuracy with different filtering ratios (Figure 7a). The results indicate that aggressive filtering degrades accuracy, leading to more than 7% drop when filtering out 90% of updates. However, filtering out 50% of updates results in only a 0.37% accuracy drop.

Sparsity. Gradient descent leverages the sparsity of BCI signals to filter out redundant parameter updates. As the human brain communicates and processes information in a highly sparse manner using spikes, the recorded signals also exhibit significant sparsity. Consequently, it is a common practice to apply a threshold to the recorded signals during inference [58, 100, 118, 133]. We extend this approach to continual learning by filtering out parameter updates induced by signals that fall below a specified threshold.

To evaluate the consequence of filtering, we analyze the trade-off between accuracy and filtering ratio (Figure 7b). When 99% of updates are filtered out, the accuracy drops by 9.73% from its original value. On the other hand, filtering out 95% of updates results in less than 0.55% accuracy degradation.

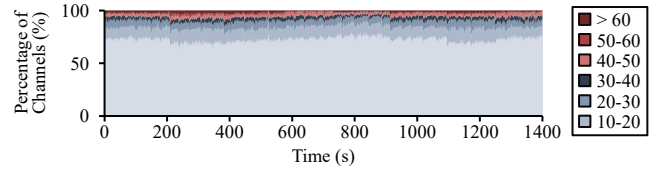


Figure 8: Firing rate distribution of channels over time.

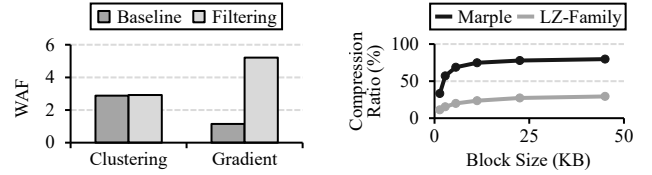


Figure 9: Write amplification in BCI learning algorithms. Figure 10: Compression ratio with different block sizes.

Key Observation #1: The recurrent and sparse nature of neural signals makes many updates redundant and ineffective.

This observation motivates us to design a filtering module that can identify the ineffective updates based on the input signals. Then we aim to integrate this module into the BCI system to eliminate redundant NVM write operations.

3.2 Locality in BCI Signals

Neural signals exhibit high temporal locality, reflecting the synchronous activity of neurons. Our analysis shows that only a small subset of channels captures the majority of active signals, as only a few neurons are highly activated during specific tasks. Therefore, we can reduce the NVM write overhead by buffering those associated with the frequently recorded channels.

We quantify the temporal locality of BCI signals by analyzing the firing rate distribution of channels over time, using neural signal data recorded from a patient imagining handwriting sentences (Figure 8). The result reveals that only 5.57% of the channels are highly active (i.e., firing rate over 40 Hz) when the patient imagines handwriting a specific sentence. This implies that a substantial portion of write operations is repeatedly directed to a localized memory region.

Key Observation #2: The temporal locality of neural signals results in frequent updates to specific memory regions.

We leverage this observation by accumulating repeated updates that targets the same memory region. Our goal is to design a low-cost buffer management scheme that minimizes NVM writes.

3.3 Write Amplification in Updates

BCI learning algorithms suffer from severe write amplification due to sparse and irregular updates, whose granularity is often smaller than the page size of commercial NVMs.

We examine the write amplification factor (WAF) when deploying representative learning algorithms on an NVM-assisted BCI

system (Figure 9). We deploy clustering and gradient descent, assuming a 4 KB page size, aligning with commercial NAND Flash standards [80]. We then compare the WAF before and after filtering out redundant updates (Section 3.1). The results show that clustering experiences a WAF of 2.88 due to its inherently sparse nature. In contrast, gradient descent does not suffer from high WAF without filtering as it updates the entire weight matrix in contiguous memory regions. However, applying filtering (Section 3.1) fragments the contiguous updates, leading to a high WAF of 5.33.

Key Observation #3: BCI learning workloads exhibit severe write amplification due to fine-grained and irregular updates.

Based on the observation, we propose to aggregate fine-grained and irregular writes into contiguous data chunks aligned with the NVM page size. Our goal is to design an efficient flushing mechanism to pack sub-page updates before writing them to the NVM.

3.4 Compression Algorithms for BCI Learning

Compression is a promising technique for optimizing NVM usage in write-intensive BCI learning workloads. Our work focuses on compressing waveform centroids (i.e., templates), which serve as representative model parameters for clustering. Unlike gradient descent, which benefits from various compression techniques (e.g., pruning), waveform compression remains relatively unexplored. Existing algorithms (i.e., LZ family [137, 138] and Marple [102]) are inefficient in this context, as they rely on large block sizes to achieve adequate compression ratios, introducing additional overhead when the entire block is recompressed for each fine-grained update.

Figure 10 illustrates the compression ratios of BCI signal waveforms across different block sizes, using a representative online compression algorithm from the LZ family [137, 138] and a specialized BCI compression algorithm, Marple [102]. While LZ-family algorithms are lossless, Marple employs lossy compression. Therefore, we evaluate the compression ratio of Marple while maintaining a relative accuracy above 99% in a spike sorting task.

The results show that the compression ratio degrades significantly as the block size decreases. LZ-family algorithms suffer from fewer opportunities to identify repeated patterns in smaller blocks. Although Marple achieves a relatively higher compression ratio by exploiting waveform characteristics, its performance also declines with smaller block sizes due to substantial metadata overhead, which becomes dominant when the block size is small.

As a result, existing compression algorithms are unsuitable for handling fine-grained updates in BCI learning workloads. Compressing in large blocks achieves high compression ratio but introduces significant overhead, as the entire block must be recompressed for each update. Conversely, using smaller block sizes to minimize the overhead severely limits the compression efficiency.

Key Observation #4: BCI learning workloads require compression with small blocks to align with local updates.

To address the limitation, we propose a local-block-based compression algorithm tailored for waveform data in BCI learning workloads. Our goal is to achieve a sufficiently high compression ratio even with small block sizes.

4 InfiniMind System

In this section, we present InfiniMind, a learning-optimized large-scale BCI system (Figure 11). We design four key hardware components based on the design goals in Section 3. First, InfiniMind utilizes a learning-aware update filtering unit to eliminate redundant writes (Section 4.1). Second, our delta buffering architecture effectively reuses modified data using a small buffer and an application-specific mapping table (Section 4.2). Third, the delta buffer employs an out-of-place flushing mechanism to reduce write amplification induced by fine-grained updates (Section 4.3). Fourth, a learning-friendly waveform compression unit further reduces write overhead by reducing the volume of data written to NVM (Section 4.4). The four components are seamlessly integrated into SCALO, a multi-accelerator NVM-assisted BCI inference system. We extend its memory controller, NoC, and PEs to orchestrate learning-induced memory access, computation, and communication patterns (Section 4.5). The resulting design, InfiniMind, supports end-to-end processing optimized for both learning and inference workloads in NVM-assisted BCI systems.

4.1 Update Filtering

We integrate an update filtering unit into the memory controller and modify the application pipeline to detect and filter out redundant write requests. The PEs in the learning pipeline embed both updated data and metadata (e.g., similarity) to NVM write requests. For each request, the filtering unit examines the metadata to assess recurrence and sparsity, and invalidates the request if deemed redundant. Figure 12 illustrates the filtering methods for clustering and gradient descent which exploit recurrence and sparsity, respectively. Each adopts a different type of metadata to determine redundant writes.

Clustering (Figure 12a). Clustering leverages the similarity between the input waveform and cluster centroids to identify redundancies. First, the PEs compute the similarity between all the cluster centroids associated with the corresponding channel to find the best matching cluster (①), and updates its centroid accordingly (②). Next, the PE forwards both the updated centroid (i.e., templates) and the similarity value to the filtering unit (③). Upon receiving the data, the filtering unit checks if the similarity is below a threshold and conditionally writes the data (④).

Gradient Descent (Figure 12b). Gradient descent leverages the sparsity of neural signals to identify redundancies. First, the PEs perform the forward and backward passes of the target network using the input signals to update the model parameters, such as weights (①). Next, they forward both the updated parameters and the original input signals to the filtering unit (②). Accordingly, the filtering unit checks if the input signal exceeds a threshold and writes the updated parameters associated with the input (③).

4.2 Delta Buffering

We implement a low-cost buffering architecture that aggregates updates from active input signals to exploit signal locality. It includes a delta buffer that caches data associated with active inputs, and a custom mapping table that supports flexible, application-aware management for the data within the buffer.

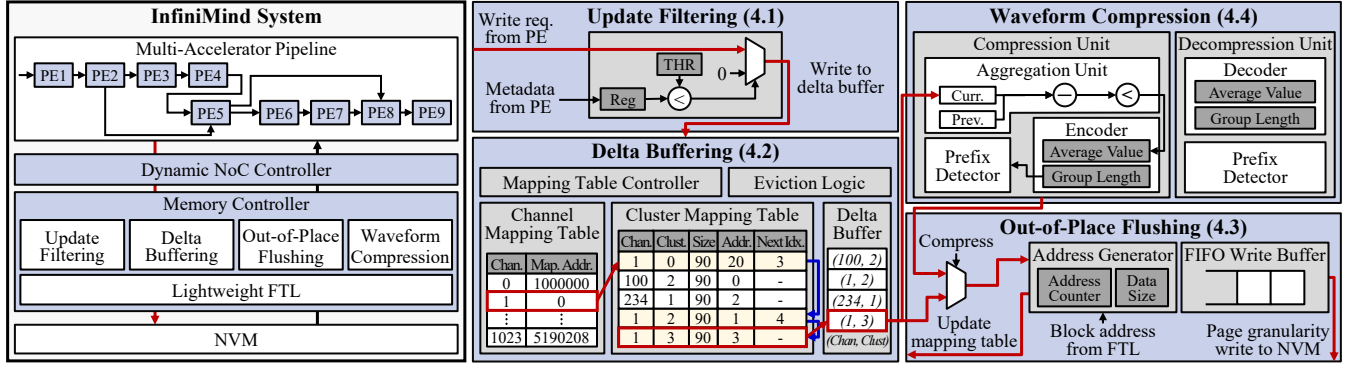


Figure 11: InfiniMind system overview.

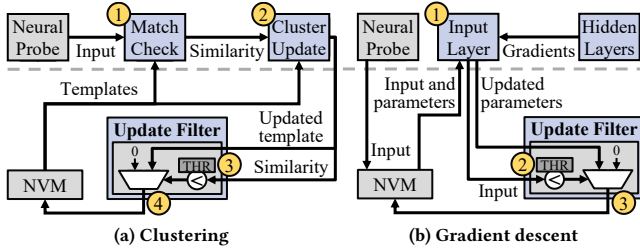


Figure 12: Filtering steps for clustering and gradient descent.

Delta Buffer. The delta buffer is a space-efficient cache architecture that retains only the modified portions of data within updated pages, avoiding the overhead of storing entire fixed-size pages. It flexibly supports application-dependent update granularity, determined by the number of parameters updated simultaneously in each application. For example, the granularity depends on the number of weights updated for a given input channel in gradient descent, or the size of a single centroid in clustering.

The buffer aggregates updates to the same memory addresses and flushes data that is unlikely to be updated when the buffer is full. We conduct a design space exploration of buffer capacities and eviction policies to maximize the buffer hit ratio under the strict power budget of the implantable system. We further elaborate on the design space exploration in Section 5.5.

Mapping Table. The buffering architecture incorporates a hierarchical mapping table to flexibly manage data within the buffer and to track buffer hits and misses. It also supports application-level indexing for dynamic data structures in BCI learning algorithms. For example, it can traverse and update linked lists to manage clusters that are dynamically created and removed during clustering. To cope with limited on-chip memory capacity, we propose an efficient caching mechanism for the mapping table.

The mapping table consists of two hierarchical levels: (1) the channel table keeps an entry per channel ID and (2) the cluster table keeps an entry per cluster ID. The cluster table organizes entries for each channel as a linked list, where each entry keeps the address of a centroid and a pointer to the next entry (i.e., next index). New entries are created or deleted by modifying the pointers. Figure 11

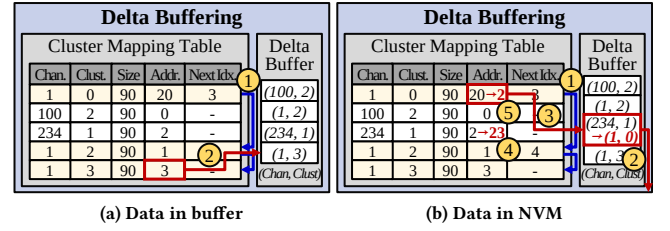


Figure 13: Write operation with delta buffering architecture.

illustrates the hierarchical mapping table structure and the linked list organization (blue arrows) for the cluster table.

However, linked lists incur significant NVM access overhead due to the scattered memory access patterns. To mitigate the overhead, the controller converts the linked list into a contiguous array when flushing the cluster table entries to the NVM. Conversely, when loading entries into the cache, the controller reconstructs the linked list to efficiently manage deletion and creation of clusters.

Figure 13 shows a write operation with the delta buffering architecture. The blue arrows illustrate an example linked list for channel 1, while the red arrows show the dataflow for the write operation. Addresses 20 and above indicate NVM, while those below 20 refer to the delta buffer. The operation begins by identifying the mapping table entry for the target (*Channel ID*, *Cluster ID*).

Figure 13a demonstrates the write mechanism for (1, 3). First, the operation locates the head of the linked list using the channel mapping table. It starts by locating the mapping entry for (1, 0) and traverses the linked list to find the target cluster ID (①). The address field is used to identify the data location, which, in this case, is the buffer at index 3, where the modified data is written (②). If the entry points to the NVM, it loads the data from the NVM, potentially triggering an eviction. Figure 13b shows an example of a write operation for (1, 0). Similarly, it searches for the mapping entry and checks the address field to determine data location (①). Here, the address is 20 in the NVM, causing an entry in the delta buffer to be evicted and flushed to the NVM (②). The data is then written to the newly available slot in the delta buffer (③). Finally, the mapping table is updated to reflect the new data locations for both the new and evicted entries. Specifically, the address for (234, 1) is updated

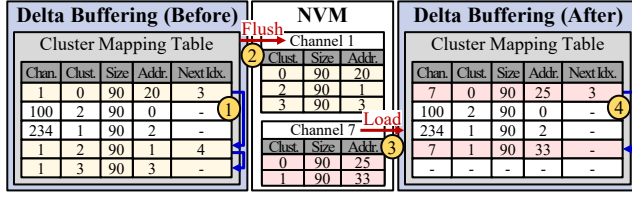


Figure 14: Caching mechanism for cluster mapping table.

to 23, indicating new NVM address (④), and the address for (1, 0) is set to 2, indicating the new location in the delta buffer (⑤).

Figure 14 illustrates the caching mechanism for the hierarchical mapping table, focusing on the cluster mapping table. The example demonstrates how entries for channel 7 are loaded from the NVM, causing the entries for channel 1 to be flushed. The left side shows the cluster table before flushing and the right side shows the table after flushing and loading. First, the linked list is traversed to extract all entries in the cluster table for channel 1 (①). These entries are then gathered into a contiguous array and written to the NVM (②). Next, the entries for channel 7 are retrieved from a contiguous region in the NVM (③). When these entries are stored in the cache, they are scattered into the slots vacated by the eviction of channel 1's entries. Finally, the linked list is reconstructed by updating the pointers within the entries (④).

4.3 Out-of-Place Flushing

We propose a flushing methodology for the delta buffer to mitigate write amplification. As the delta buffer operates on sub-page granularity, flushing each entry incurs sub-page NVM writes. To mitigate the overhead, we propose an out-of-place flushing method motivated by the log-structured file system (LFS) [95]. The key idea of LFS is to buffer random NVM writes into a contiguous memory region to minimize write overhead. In a similar manner, delta buffer allocates a new page and packs multiple sub-page updates into a single page. After that, it updates the mapping table entries to index the flushed data at the new page address. Note that our mapping table inherently supports sub-page granularity updates, enabling efficient and seamless integration of out-of-place flushing.

We extend the delta buffer to flush entries to new memory addresses using an address generator and a FIFO write buffer (Figure 11). When delta buffer flushes an entry, the address generator requests a free block address from the FTL, generates a physical page address, and assigns the address to the entry. Then, it increments the address counter by the size of the flushed entry and uses the counter to assign a new address for the subsequent flush operation. If the counter exceeds a single block size, it requests a new physical block address and resets the counter. As the address generator enforces sequential writes to a contiguous memory region, the FIFO write buffer can pack the writes into page-sized chunks. It holds up to two pages, which is sufficient to buffer sub-page writes.

4.4 Waveform Compression

We propose a learning-friendly waveform compression algorithm with hardware support to further reduce NVM writes. Our custom algorithm achieves a high compression ratio even with a small

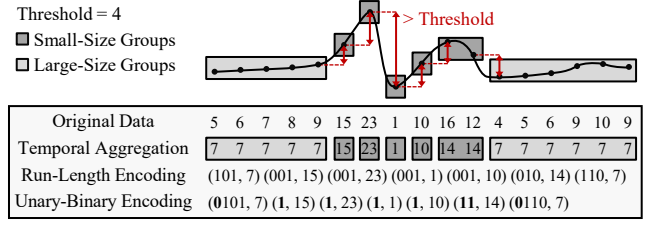


Figure 15: Waveform compression mechanism.

block size. Therefore, it avoids extra writes due to the fine-grained parameter updates in clustering. The key idea is to (1) classify the waveform samples into stable and active parts and (2) apply lossy compression to the stable parts while minimally sacrificing waveform quality. We also implement a custom hardware module for fast compression and decompression at a low cost.

Our compression algorithm consists of two steps: (1) aggregating consecutive samples with similar values and (2) compressing them using unary-binary encoding. Figure 15 illustrates the overall compression mechanism using an example waveform, with two unary bits and three binary bits used for encoding. Dark and light boxes represent temporally aggregated groups, and boldface numbers denote the prefix bits generated by unary-binary encoding.

The algorithm begins with lightweight aggregation, sequentially scanning the waveform samples and assigning consecutive samples to different groups if their difference exceeds a predefined threshold. As a result, samples in stable parts of the waveform are categorized into a single large-size group, while those in active parts are split into smaller, independent groups. For instance, in Figure 15, the difference between the fifth and sixth samples is six, which exceeds the threshold of four, thereby placing them into separate groups. After scanning all samples, each group is assigned the average value of its elements. We then apply run-length encoding to represent each group as (Group length, Average value).

We further compress the data using unary-binary encoding [1]. We use unary encoding for small group length, reducing bits for frequent small-sized groups. For larger groups, we employ binary encoding with a prefix zero. This is highly effective because samples from active regions form many small groups, while those from stable regions form a few large groups.

Our BCI-specific compression leverages local information of a single waveform. Therefore, it achieves a high compression ratio even with small blocks, making it suitable for BCI learning workloads. In a detailed analysis using spike sorting, a representative clustering algorithm, we demonstrate sufficient compression ratio with small block sizes aligned to the template size (i.e., 1.4 KB), avoiding additional write overhead. The method achieves 82.75% compression ratio with 99.9% relative accuracy.

To efficiently deploy the algorithm, we design a dedicated hardware module that performs compression and decompression for write and read operations on waveform data, respectively (Figure 11). The compression unit includes an aggregation unit, an encoder, and a prefix generator, each for the steps of temporal aggregation, run-length encoding, and unary-binary encoding. The decompression unit includes a prefix detector and a decoder to reverse the compression process and restore the original data.

Table 1: PE configurations.

Name	Clock (MHz)	Leakage (μ W)	Dynamic (μ W)	Total (μ W)	Area (mm^2)	Latency (μ s)
BBF	41.67	101.68	612.12	713.80	0.05314	0.096
MAD	60	325.45	1322.11	1647.56	0.08941	17.07
DTW	100	43.27	249.09	292.36	0.0097	3.60
NEO	20.83	13.36	17.05	30.40	0.00084	0.048
THR	20.83	1.87	1.33	3.19	0.00008	0.048
BIN	41.67	54.70	84.12	138.82	0.05931	0.024
VEC	60	28.58	45.02	73.59	0.00066	0.017
GTH	100	989.42	3299.97	4289.40	0.07745	11.20

4.5 End-to-End BCI Learning System

We implement a learning-optimized NVM-assisted BCI system by applying four optimization schemes to SCALO. To seamlessly integrate the hardware modules for learning, we modify the memory controller, PEs, and NoC (Section 4.5.1). Next, we present an end-to-end processing pipeline of InfiniMind, demonstrating its capabilities for both learning and inference (Section 4.5.2). In addition, we describe how to configure the system to achieve the desired latency, power consumption, and accuracy (Section 4.5.3).

4.5.1 Learning-Enabled NVM-Assisted BCI System. To efficiently deploy learning algorithms on SCALO, we modify three key components. First, we implement a lightweight hardware-based FTL to efficiently handle garbage collection and wear leveling, both critical for write-intensive learning workloads. Second, we redesign the processing pipeline to support learning-specific computation patterns. Finally, we extend the NoC to manage variable-latency data movement and to alleviate underutilization of processing pipelines. **Lightweight FTL.** We design a lightweight, hardware-based FTL to efficiently manage NVM at a low cost. Conventional FTLs rely on CPUs and DRAMs, exceeding the power constraints of implantable devices. For example, our experiment using an FTL implemented with an in-order CPU and low-power DRAM reveals that the FTL alone consumes 57 mW. To address this, we implement a custom hardware-based FTL using a lightweight hardware module integrated with small on-chip SRAMs, which supports garbage collection and wear leveling. Regarding logical-to-physical address translation, we integrate it into the application-specific mapping table in the delta buffering architecture (Section 4.2). This eliminates the need for address translation in the FTL, reducing memory overhead and minimizing update costs during data writes.

The controller employs a greedy garbage collection strategy [12, 18] and dynamic wear leveling mechanism [70]. To support these with out-of-place flushing (Section 4.3), it maintains three types of metadata: (1) valid byte counts per page, (2) invalid page counts, and (3) a free block list sorted by erase counts. During each out-of-place flushing, the controller decrements the valid byte counts for the original pages and increments the invalid page count when a byte count becomes zero. If the address generator requests a new physical block, the controller selects the block with the lowest erase count to minimize wear imbalance. To efficiently manage metadata, the controller caches only a small portion in SRAM, while storing the rest in NVM. This simple mechanism is effective for BCI learning workloads with a relatively small working set size.

Learning-Optimized Processing Pipeline. Table 1 lists the PEs in the InfiniMind system. We implement BBF, MAD, DTW, NEO, and

THR based on the descriptions in SCALO [108] (and HALO [53]). Each PE supports dedicated computational kernels, including band-pass filtering (BBF), matrix-vector multiplication (MAD), dynamic time warping (DTW), non-linear energy operator (NEO), and thresholding (THR). To support learning, we implement additional PEs including binning (BIN), vector operation (VEC), and gathering (GTH). BIN accumulates input data over a predefined period and conditionally applies Gaussian smoothing via a sliding window approach [33]. VEC performs element-wise vector operations (e.g., addition, subtraction, multiplication) as well as various auxiliary functions (e.g., ReLU, tanh, sigmoid, and their derivatives). GTH buffers input and gathers the correlated data into chunks. For example, it gathers inputs from neighboring channels and transfers them to the next PEs upon detecting a spike. Section 4.5.2 details how these PEs are utilized for learning.

SCALO is a globally-asynchronous locally-synchronous (GALS) architecture where each PE operates at its optimal clock frequency, with inter-PE communication managed by a circuit-switched NoC and FIFO queues. The PEs are fully pipelined, allowing them to operate at low clock frequencies while maintaining high throughput and low latency. However, the pipelined design suffers from underutilization due to data hazards between inference and learning stages, as well as structural hazards associated with NVM access. Figure 16 illustrates the underutilization using the spike sorting pipeline as an example (shown in Figure 19a and explained in Section 4.5.2). First, PEs are underutilized due to read-after-write (RAW) data dependencies, as the PEs must complete learning for the previous timestep before proceeding inference for the next timestep (Figure 16a). Second, only a single PE can access the NVM at a time, which causes other PEs that require NVM access to stall until it becomes available (Figure 16b).

These underutilization issues are inherent in learning-enabled workloads. Therefore, we design the processing pipeline to operate in a serial manner, where only one PE is active at a time (Figure 17). InfiniMind employs a dedicated controller to selectively activate a single PE while clock-gating the others to minimize power consumption. Although this approach requires a larger FIFO queue to buffer intermediate data before activating the next PE, our experiments indicate that the resulting power savings outweigh the associated overhead.

To mitigate the increased processing latency introduced by serial execution, we further optimize the PEs for low-latency operation. First, we redesign the PE microarchitecture to prioritize latency over throughput. Second, since only one PE is active at a time, we operate each PE at a higher clock frequency, leveraging the power savings gained from clock-gating in the serial execution pipeline.

Dynamic NoC. We design a dynamic NoC by extending the low-power circuit-switched NoC in SCALO to support two distinct communication patterns: (1) PE-to-PE communication and (2) PE-to-NVM communication.

First, it mitigates the underutilization issue in learning pipeline by reusing the PEs. As shown in Figure 17, InfiniMind’s processing pipeline still suffers from significant underutilization. However, under a sequential execution model, we observe that the PEs can be reused across different computation stages to improve utilization (e.g., the two THR stages in Figure 17). To support this, we implement a dynamic NoC controller that can temporally fold the

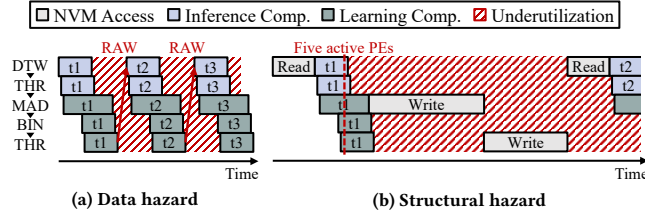


Figure 16: PE underutilization in processing pipelines.

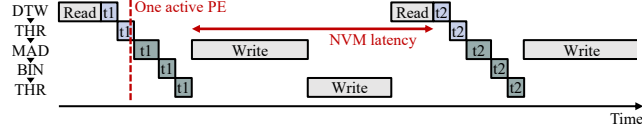


Figure 17: InfiniMind's sequential execution model.

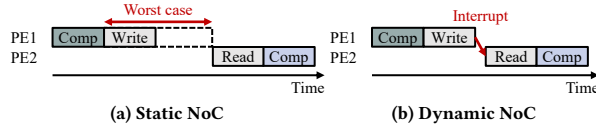


Figure 18: Timeline for supporting variable-latency NVM accesses via static and dynamic NoC.

processing pipeline. The dynamic NoC controller is programmed with a sequence of computation stages for each application and advances stages upon receiving interrupt signals from the PEs. Based on the current computation stage, it reconfigures the NoC connections to supply operands to the PEs. Additionally, it transmits control signals to the PEs via a handshake synchronizer. The PE parameters are then reconfigured (e.g., threshold) based on the signal from the dynamic NoC controller, allowing a single PE to time-multiplex computations originally assigned to separate PEs.

Second, it efficiently manages variable-latency NVM accesses (Figure 18). In the target pipeline, the data is processed by PE1, written to NVM, read from NVM to PE2, and then processed by PE2. A static circuit-switched NoC maintains a connection between PE1 and NVM for a fixed duration (Figure 18a). Therefore, the NoC waits for the worst-case latency before switching connections to PE2–NVM, limiting the benefits of the write latency reduction techniques (e.g., delta buffering). In contrast, the dynamic NoC controller reconfigures the connection upon receiving an interrupt signal (Figure 18b). Rather than waiting for the worst-case latency, it immediately determines the next PE and switches to the PE2–NVM connection to initiate read operation.

4.5.2 End-to-End Processing Pipeline for BCI Learning. Figure 19 presents two examples of end-to-end processing pipelines for deploying representative BCI learning workloads on InfiniMind. Each pipeline includes preprocessing, inference, and learning stages.

Clustering (Figure 19a). We employ spike sorting as an example clustering workload. Spike sorting identifies which neurons have fired when a spike is detected in a recording channel. The example pipeline implements OSort, a representative learning-enabled spike sorting algorithm [98]. In preprocessing, BBF filters input signals,

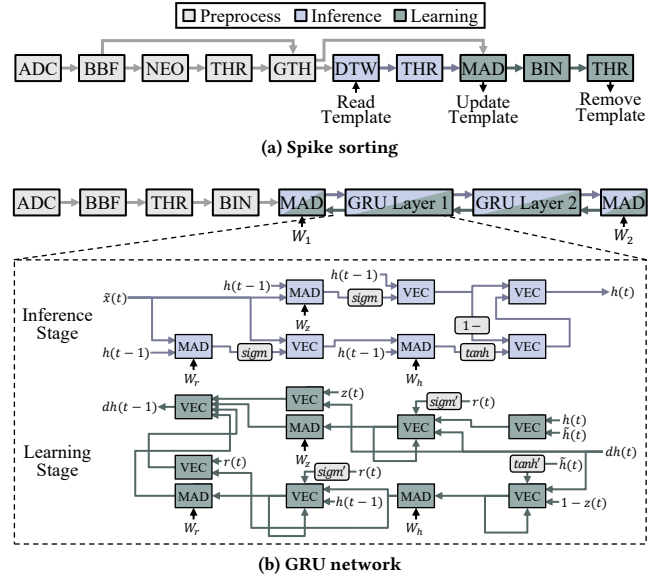


Figure 19: End-to-end processing pipelines with learning support on InfiniMind.

and NEO and THR detect spikes from recording channels. When a spike is detected in a channel, GTH collects signals from adjacent channels. During inference, DTW retrieves cluster centroids (i.e., templates) and compares them with input signals. If the similarity exceeds a predefined threshold (THR), MAD updates the template; otherwise, it creates a new one. MAD sends write requests with the updated template and similarity metadata to enable update filtering at the memory controller. Periodically, BIN and THR remove infrequently matched templates as described in Section 2.2.

Gradient Descent (Figure 19b). For the gradient descent workload, we implement a gated recurrent unit (GRU) network, a representative neural network model for character decoding [26, 35]. In preprocessing, BBF filters input signals, THR detects peaks, and BIN aggregates data into bins. During inference, MAD reads weight matrices from NVM, computes matrix-vector products, and forwards results to PEs for subsequent layers. It also stores the intermediate activations in NVM for backpropagation during learning. In the learning stage, MAD calculates gradients and backpropagates them through the network. It reads intermediate activations from NVM, computes gradient-vector products, and updates weight matrices. Note that InfiniMind dynamically reuses PEs (e.g., MAD, VEC) across layers and between inference and learning stages via its dynamic NoC to optimize power and area.

4.5.3 Configuring InfiniMind. Before deploying the target application, we use software to configure the InfiniMind system based on latency and power constraints, as in SCALO. However, our approach also considers accuracy and iteratively performs the following steps until it finds an optimal configuration. First, the software determines the minimum clock frequency of the PEs based on the number of electrodes, sampling frequency, and processing pipeline. It then sets the compression threshold, based on our observation

that the optimal threshold scales with the sampling frequency. Next, it determines the maximum NVM bandwidth that meets the power constraint. Finally, it profiles the workload under various filtering thresholds to find the configuration with minimal accuracy drop within the latency budget. If accuracy drops significantly, the software adjusts the number of electrodes and sampling frequency, then repeats the procedure. However, runtime deployment may deviate from profiling results, which may lead to accuracy degradation or latency violations. In such cases, InfiniMind dynamically reconfigures the software-defined filtering threshold to adapt to runtime conditions.

5 Evaluation

5.1 Experimental Setup

InfiniMind processes signals recorded from the state-of-the-art large-scale neural recording device [130]. The platform consumes up to 24.7 mW when recording signals from 1,024 channels at 20 KHz sampling rate. We assume a strict power budget of 45 mW [94] to prevent overheating at the implant site. To evaluate the digital components, we implement the system in Verilog and generate SRAMs using the open-source OpenRAM memory compiler [43]. Logic synthesis and power analysis are performed with Cadence tools, using the 45-nm NanGate FreePDK45 Library [110]. We then scale the technology node from 45 nm to 28 nm to integrate InfiniMind with SCALO. For end-to-end performance evaluation, we develop a custom trace-driven, cycle-level simulator integrated with the open-source solid state drive (SSD) simulator, SimpleSSD [52]. Flash memory operations are modeled using timing parameters and electrical characteristics from a commercial single-level cell (SLC) NAND Flash datasheet [80].

We implement the delta buffer with a 72 KB SRAM. The channel mapping table uses a 4 KB SRAM with 32-bit entries pointing to the cluster mapping table. The cluster mapping table employs a 16 KB SRAM with 64-bit entries, each embedding a channel ID, cluster ID, size, address, and next index. Additionally, an 8 KB SRAM is allocated to the FIFO write buffer. For NVM, we use a 1GB SLC NAND Flash with a 4 KB page size and a 256 KB block size.

We evaluate InfiniMind using four benchmarks to demonstrate its effectiveness in diverse BCI learning applications. Specifically, we use a gated recurrent unit network (GRU) and a multi-layer perceptron (MLP) for gradient descent, and employ spike sorting (SS) and template matching (TM) for clustering. Due to the limited availability of large-scale BCI datasets, we scale the open-source datasets to match our intended deployment. We divide datasets into time chunks and spatially concatenate them to create the 1,024-channel dataset and resample to a 20 KHz sampling rate.

For GRU, we use a character decoding dataset of neural recordings from a human performing a handwriting task [120]. This dataset contains neural activity from two 96-channel microelectrode arrays, binned into 20 ms intervals. The model comprises an input fully connected (FC) layer, a two-layer GRU network, and an output FC layer. To handle temporal variations in neural signals, the input layer is continuously recalibrated during operation [35]. After decoding each word, the model is recalibrated by backpropagation through time (BPTT) over the word sequence. For real-time calibration, the entire backpropagation process must complete within

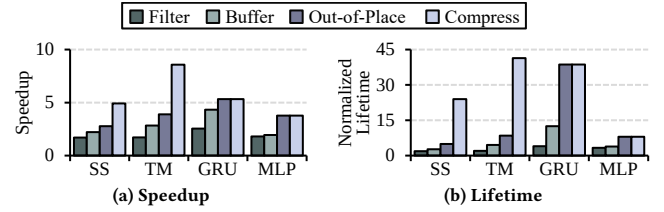


Figure 20: Speedup and lifetime improvements achieved by each optimization scheme.

800 ms, the average time between finishing a word and beginning to write the next, based on human handwriting data.

For MLP, we leverage the MAZE dataset from the Neural Latents Benchmark [91], which records neural activity from two 96-channel Utah arrays implanted in the primary motor cortex of a monkey performing a reaching task. We bin the data into 40 ms windows and process it using a three-layer multi-layer perceptron (MLP) model. During learning, model weights are updated to adapt to dynamically changing neural signals. Each inference phase is followed by a learning phase, requiring the system to complete both phases within the 40 ms bin size for real-time operation.

For SS, we use the SpikeForest hybrid_drift_siprobe dataset [77], which includes 64-channel neural recordings from a Neuropixels probe with simulated electrode drift of 10 μ m amplitude. We apply a multichannel spike sorting algorithm modified from OSort [86]. For real-time performance, the system must complete inference and learning within the average neuron firing interval (i.e., 0.18 ms).

For TM, we use seizure detection data from the UPenn and Mayo Clinic Seizure Detection Challenge [5]. Specifically, we use Patient 1's 68-channel neural recordings spanning 70 seconds of ictal and 104 seconds of interictal phases. Templates are generated by averaging seizure-classified waveforms in the training set using a 25 ms window, matched with input signals for detection and recalibration. The system must complete inference and learning within the 25 ms window for real-time performance.

5.2 Benefits of the InfiniMind System

We evaluate the performance and lifetime benefits of the four key techniques adopted in InfiniMind. As a baseline, we utilize the learning-enabled NVM-assisted BCI system (Section 4.5.1) without our optimization schemes (Baseline). We then incrementally apply update filtering (Filter), delta buffering (Buffer), out-of-place flushing (Out-of-Place), and waveform compression (Compress).

The results demonstrate that each optimization greatly improves the performance of the BCI system (Figure 20a). The four techniques contribute to a 1.91 $\times$, 1.41 $\times$, 1.43 $\times$, and 1.98 $\times$ speedup, respectively. As a result, InfiniMind achieves an average of 5.39 $\times$ speedup over Baseline. Note that we apply Compress exclusively to clustering workloads (i.e., SS and TM) as it is designed to compress waveforms.

Similarly, each optimization enhances the lifetime of the NVM by reducing the number of NVM write operations (Figure 20b). The four techniques improve NVM lifetime by 2.65 $\times$, 1.86 $\times$, 2.16 $\times$, and 4.88 $\times$, respectively. Overall, InfiniMind extends the NVM lifetime by 23.97 $\times$, 41.40 $\times$, 38.67 $\times$, and 7.98 $\times$ for each benchmark and achieves 3.12, 0.83, 15.56, and 0.95 years of lifetime, respectively.

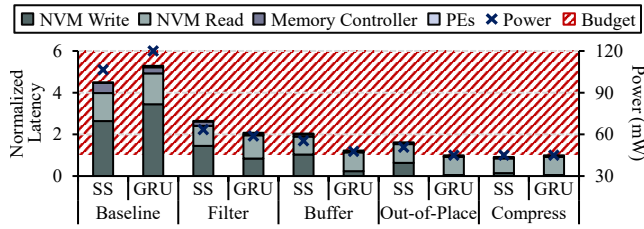


Figure 21: Latency breakdown and power consumption for each optimization step.

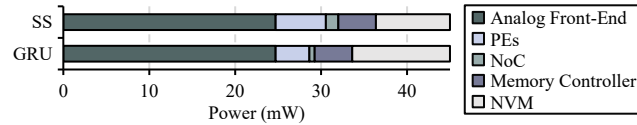


Figure 22: Power breakdown for BCI learning workloads.

5.3 Latency Analysis

We analyze the latency of InfiniMind to highlight the speedup contributed by each optimization scheme and to demonstrate that InfiniMind satisfies the real-time requirements of BCI applications (Figure 21). As benchmarks, we profile SS and GRU, the most representative workloads for clustering and gradient descent, respectively. Latency is categorized into four components: NVM Write, NVM Read, Memory Controller, and PE.

Update Filtering. By filtering out 50% and 95% of updates for SS and GRU, we significantly reduce NVM write latency by 1.82 $\times$ and 4.10 $\times$, respectively. The filtering also decreases read latency by 1.40 $\times$ and 1.32 $\times$, respectively, as filtering redundant updates reduces both read and write operations. Additionally, it minimizes memory controller latency by lowering garbage collection overhead resulting from fewer write operations.

Delta Buffering. Buffer further reduces NVM write latency by 1.40 $\times$ for SS and 3.54 $\times$ for GRU, by reusing 17% and 71% of buffered data, respectively. The buffering mechanism is particularly effective for GRU, which updates data at finer granularity and exhibits higher temporal locality. We further discuss buffer hit ratios in Section 5.5.

Out-of-Place Flushing. Out-of-Place reduces the WAF by 60% for SS and 78% for GRU, lowering NVM write latency by 1.62 $\times$ and 4.50 $\times$, respectively. GRU achieves greater improvement because it updates data at finer granularity after filtering, making it more susceptible to write amplification (Section 3.3).

Waveform Compression. Compress achieves 4.50 $\times$ lower NVM write latency for clustering (SS) with only a 3.6 μ s latency overhead. Additionally, Out-of-Place cooperatively reduces latency, mitigating the write amplification caused by compression.

5.4 Power and Area Analysis

We evaluate the power consumption required to meet the latency budget on SS and GRU, by cumulatively applying the optimization schemes (Figure 21). Note that to meet the latency budget for the baselines, we allocate a higher bandwidth than specified in the datasheet [80]. Also, we break down the power consumption of the final design (Figure 22). The results confirm that InfiniMind

Table 2: Additional hardware components in InfiniMind.

Name	Clock (MHz)	Leakage (μ W)	Dynamic (μ W)	Area (mm^2)
Update Filtering	100	3.45	7.31	0.00018
Delta Buffering		260.40	2582.06	0.31390
Out-of-Place Flushing		24.28	714.68	0.02734
Waveform Compression		8.26	21.81	0.00037
Lightweight FTL		31.57	722.22	0.03269
Dynamic NoC Controller		2.94	8.61	0.00016
Total		330.89	4056.69	0.37465

operates within the power budget by limiting the NVM bandwidth according to the available power budget. For SS, the power consumption of the analog front-end, PEs, NoC, and memory controller is 24.7 mW, 5.85 mW, 1.45 mW, and 4.38 mW, respectively. This allows NVM to consume 8.62 mW, resulting in read and write bandwidths of 627.54 Mbps and 78.44 Mbps, respectively. In contrast, GRU consumes 3.94 mW of power on the PEs, leaving more power budget for the NVM. The NVM consumes up to 11.38 mW, resulting in 828.56 Mbps read and 103.57 Mbps write bandwidth.

Table 2 summarizes the power consumption and area of the InfiniMind’s additional hardware components integrated into the baseline system. The entire system excluding NVM occupies a total area of 1.24 mm^2 , with the PEs and NoC accounting for 0.71 mm^2 and 0.15 mm^2 , respectively. The hardware for the four proposed schemes, the FTL, and the dynamic NoC controller collectively account for 0.37 mm^2 , resulting in a 30.33% area overhead.

5.5 Sensitivity Tests

We analyze the impacts of various design choices, including eviction policy, batch size, compression ratio, and learning accuracy.

Eviction Policy. Figure 23 shows the sensitivity of the delta buffering scheme to buffer size and eviction policy. We evaluate hit ratios for buffer sizes from 18 KB to 144 KB, comparing Least Recently Used (LRU) and Least Frequently Used (LFU) eviction policies. The results show that LFU outperforms LRU, as LRU frequently evicts hot entries due to the rotating write patterns across channels. For instance, GRU achieves a 39.58% hit ratio with LFU and 17.02% with LRU at a 36 KB buffer size (Figure 23). The hit ratio discrepancy between workloads stems from variations in BCI signal locality and working set sizes (2.4 MB vs. 227 KB). Our choice is a 72 KB buffer size with an LFU eviction policy.

Batch Size. Many gradient descent-based learning algorithms use batching to reduce computational overhead. InfiniMind can also benefit from batching to achieve additional speedup, as shown in Figure 24. With a larger batch size, fewer NVM writes occur, as gradients for multiple inputs are accumulated on PE SRAMs. However, batching yields a smaller speedup for GRU than MLP since NVM writes account for only 5.31% of GRU’s total latency after all optimizations. Speedup saturates at four because the number of inputs that can be processed at once is limited by the SRAM size.

Compression Ratio. Because of the lossy nature of our learning-friendly waveform compression scheme, we evaluate the compression ratio and accuracy trade-off (Figure 25). Specifically, we assess the accuracy of the compressed waveform templates using the spike sorting benchmark, dynamically compressing and decompressing the waveform templates during updates. Our scheme compresses

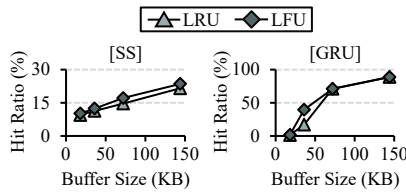


Figure 23: Delta buffer hit ratio by buffer size and eviction policy.

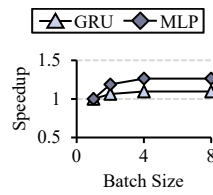


Figure 24: Speedup by batch size.

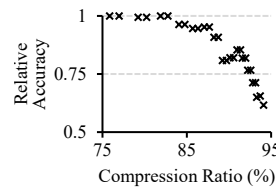


Figure 25: Compression ratio and accuracy trade-off.

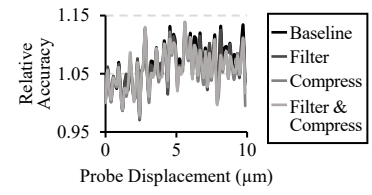


Figure 26: Accuracy drop due to filtering and compression.

the waveform templates into 17.25% of their original size, with an accuracy drop under 0.1%. However, at higher compression ratios, accuracy drops significantly, as shown in the figure.

Learning Accuracy. Figure 26 illustrates the accuracy of SS with our filtering and compression schemes (Filter and Compress) compared to the baseline. The results show that these schemes maintain accuracy over time despite non-stationarities in the neural signals.

6 Discussion & Related Work

Inference-Optimized Processors for BCIs. To meet the tight latency and power constraints, recent studies propose dedicated processors for BCI applications. Some works design accelerators tailored to specific workloads [112, 116, 135]. For example, uBrain [125] employs unary computing to accelerate DNN inference, Noema [1] introduces template-matching hardware, and Marple [102] presents an accelerator for spike sorting. To support diverse workloads flexibly, HALO [53] proposes a multi-accelerator-based inference system, which SCALO [108] extends by integrating NVM to handle larger and more complex BCI workloads. Also, NeuroLobe [67] leverages a neuromorphic processor to flexibly support BCI workloads in larger scale. However, none of these studies address the challenges of supporting learning in BCIs.

Learning-Enabled BCIs. The importance of continual learning in BCIs has been recognized in recent neuroscientific studies and clinical use cases [49, 66, 107, 129]. Some studies utilize CPU/GPU to flexibly support learning algorithms [35, 120]. However, they incur high latency and power consumption. For better performance, prior studies propose accelerators with SRAM-only memory systems to enable on-device learning [9, 79]. For example, SOUL [27] proposes an accelerator designed for gradient descent-based learning in seizure detection, and Geo-OSort [24] enables online clustering for spike sorting. However, they are tailored for specific learning algorithms, and their SRAM-only systems suffer from high power consumption, limiting their use in large-scale BCIs.

Learning Timescales. While learning is emerging as an important challenge in BCI systems, there is no consensus on the optimal frequency of learning. Some studies emphasize the need for online learning to address sudden and frequent non-stationarities [27, 35], while others adopt a more relaxed timescale (e.g., daily) [76, 120]. InfiniMind is primarily designed for the most aggressive scenarios, where learning occurs in real time for every incoming sample. It also supports batching to efficiently support learning over short timescales with a small number of samples (Section 5.5).

However, as learning timescale increases, learning from every sample may lead to excessive memory overhead due to the growing amount of data processed in a single learning step. InfiniMind

can incorporate subsampling techniques to support learning over longer timescales, such as seconds or minutes. Specifically, reservoir sampling can be employed to maintain a fixed-sized buffer of subsampled training data [2, 3, 62, 72]. As new data streams in, the sample set is continuously updated based on specific policies, enabling learning over extended timescales while keeping memory overhead constant. To integrate reservoir sampling into InfiniMind, the memory controller should be extended with additional functionalities. First, it should track the intermediate data (e.g., activations) associated with each input sample and update them whenever a sample is replaced in the subsampled dataset. Second, it should incorporate specialized sampling logic to determine which samples to replace, ensuring efficient learning while maintaining accuracy.

On/Off Hybrid Learning. This work targets on-device learning for large-scale BCIs. However, off-device learning remains critical in supporting complex training algorithms by mitigating the tight power and lifetime constraints of on-device learning. It enables training of complex models on large datasets [120] and supports advanced optimization techniques (e.g., quantization, pruning) to deploy accurate and lightweight models on-device [104]. It is also preferable for longer timescales, such as days or months, as it relieves the on-device system of excessive memory overhead.

By integrating InfiniMind into a hybrid BCI system that combines on- and off-device learning, we can leverage the benefits of both approaches. For instance, when deploying a neural network, the hybrid system can utilize InfiniMind for online learning to maintain robustness against sudden and frequent non-stationarities. At the same time, it can intermittently retrain the network using advanced off-device training methods [30, 86, 121] and synchronize the updated parameters with InfiniMind. Another possible approach is to distribute learning across the system, training the first few layers on-device while periodically updating the deeper layers off-device, as demonstrated in Internet of Things (IoT) applications [48, 50, 99].

However, integrating InfiniMind into a hybrid system introduces several architectural challenges. First, training data must be continuously transmitted to the external system via wireless communication, concurrently with on-device processing. This further constrains the power budget for on-device processing. To address this, the system should integrate a specialized compression unit to minimize communication overhead while preserving training quality. Second, the hybrid system should efficiently synchronize parameters between on- and off-device systems. If inference is stalled during parameter updates, real-time operation may be disrupted due to excessive communication and NVM write latencies. This can be mitigated by supporting background parameter updates

while allowing inference to continue. However, interleaving updates with inference may lead to inconsistent model states if only a subset of parameters is updated at any given time, potentially degrading accuracy. One solution is to maintain a shadow copy of model parameters for inference while updating an active copy in the background, synchronizing them once all updates are complete. We can leverage InfiniMind's mapping table controller to manage both copies of model parameters. However, this increases memory overhead, not only for model parameters but also for the mapping table entries required to track both copies. This overhead can be mitigated with an efficient caching mechanism that distinguishes between read-only and write-only data.

NVM Write Optimization. Many studies propose NVM write optimization strategies to enhance both performance and device lifetime across diverse applications, including databases, intermittent computing, and general-purpose computing [8, 23, 29, 89]. In this work, we adapt representative optimization techniques and integrate them into a unified system for low-power, end-to-end BCI processing. For example, the delta buffering mechanism extends page-level [51, 88] and block-level [63] write buffer management, while the flushing mechanism is inspired by the log-structured file system [95]. To support delta buffering and flushing at a low cost, we extend the mapping table to integrate BCI application-specific data management with address translation. In addition, our waveform compression mechanism extends widely adopted methods (e.g., run-length encoding), but with higher compression rates by leveraging the unique characteristics of BCI signals.

Emerging NVMs. Various NVM technologies are being developed to overcome the limitations of NAND Flash [10, 21, 55]. In this work, we focus on NAND Flash, as emerging NVMs currently lack the scalability required for large-scale BCI systems. However, future advancements may make these technologies viable by addressing existing limitations. Notably, emerging NVMs can reduce write amplification by supporting fine-grained, byte-addressable access [36, 92], while also offering faster access speeds and higher endurance compared to NAND Flash. Nevertheless, excessive write operations remain a major challenge in BCI learning workloads, which frequently update model parameters to continuously adapt to non-stationary neural signals. Therefore, regardless of the underlying NVM technology, our filtering, delta buffering, and compression techniques are applicable for mitigating write-induced bottlenecks.

7 Conclusion

In this paper, we present InfiniMind, an efficient end-to-end BCI system that supports both learning and inference. InfiniMind addresses the critical challenges when deploying write-intensive learning pipelines into NVM-assisted large-scale BCI systems. We propose four optimization schemes to minimize NVM writes based on the unique characteristics of the BCI workloads, thereby enhancing system performance and extending device lifetime. We seamlessly integrate these optimizations into SCALO by modifying its memory controller, PEs, and NoC to handle BCI learning workloads. The resulting design, InfiniMind, achieves real-time processing capabilities while satisfying the power budget of implantable BCI systems. It also demonstrates improved device lifetime by minimizing write-induced wear on the NVM.

Acknowledgments

This work was supported by Samsung Research Funding & Incubation Center for Future Technology of Samsung Electronics under Project Number SRFC-IT2501-04, and by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean Government (MSIT) (No.RS-2024-00402898, Simulation-based High-speed/High-accuracy Data Center Workload/System Analysis Platform). We also appreciate the support from Automation and Systems Research Institute (ASRI) and Inter-university Semiconductor Research Center (ISRC) at Seoul National University. The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

References

- [1] Ameer MS Abdelhadi, Eugene Sha, Ciaran Bannon, Hendrik Steenland, and Andreas Moshovos. 2021. Noema: Hardware-efficient template matching for neural population pattern detection. In *Proc. 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [2] Rahaf Aljundi, Min Lin, Baptiste Goujoud, and Yoshua Bengio. 2019. Gradient based sample selection for online continual learning. *Advances in neural information processing systems* 32 (2019).
- [3] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. 2021. Rainbow memory: Continual learning with a memory of diverse samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8218–8227.
- [4] James C Barrese, Naveen Rao, Kaivon Paroo, Corey Triebwasser, Carlos Vargas-Irwin, Lachlan Franquemont, and John P Donoghue. 2013. Failure mode analysis of silicon-based intracortical microelectrode arrays in non-human primates. *Journal of neural engineering* 10 (2013).
- [5] bbrinkm, sbaldassano, and Will Cukierski. 2014. UPenn and Mayo Clinic's Seizure Detection Challenge. <https://kaggle.com/competitions/seizure-detection>. Kaggle.
- [6] Antal Berényi, Zoltán Somogyvári, Anett J Nagy, Lisa Roux, John D Long, Shigeyoshi Fujisawa, Eran Stark, Anthony Leonardo, Timothy D Harris, and György Buzsáki. 2014. Large-scale, high-density (up to 512 channels) recording of local circuits in behaving animals. *Journal of neurophysiology* 111 (2014).
- [7] Sergio López Bernal, Alberto Huertas Celdrán, Gregorio Martínez Pérez, Michael Taynnan Barros, and Sasitharan Balasubramaniam. 2021. Security in Brain-Computer Interfaces: State-of-the-Art, Opportunities, and Future Challenges. *ACM Comput. Surv.* 54 (2021).
- [8] Abhishek Bhattacharyya, Abhijith Somashekhar, and Joshua San Miguel. 2022. NvMR: non-volatile memory renaming for intermittent computing. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*.
- [9] Sizhen Bian, Pixi Kang, Julian Moosmann, Mengxi Liu, Pietro Bonazzi, Roman Rosipal, and Michele Magno. 2024. On-device Learning of EEGNet-based Network For Wearable Motor Imagery Brain-Computer Interface. In *Proceedings of the 2024 ACM International Symposium on Wearable Computers*.
- [10] Jalil Boukhobza, Stéphane Rubini, Renhai Chen, and Zili Shao. 2017. Emerging NVM: A survey on architectural integration and research challenges. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 23, 2 (2017).
- [11] Anna Buttfeld, Pierre W Ferrez, and Jd R Millan. 2006. Towards a robust BCI: error potentials and online learning. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 14 (2006).
- [12] Werner Bux and Ilias Iliadis. 2010. Performance of greedy garbage collection in flash-based solid-state drives. *Performance Evaluation* 67 (2010).
- [13] György Buzsáki. 2004. Large-scale recording of neuronal ensembles. *Nature neuroscience* 7 (2004).
- [14] György Buzsáki, Costas A Anastassiou, and Christof Koch. 2012. The origin of extracellular fields and currents—EEG, ECoG, LFP and spikes. *Nature reviews neuroscience* 13 (2012).
- [15] Alexander J Casson. 2019. Wearable EEG and beyond. *Biomedical engineering letters* 9, 1 (2019).
- [16] João Castelano, Isabel Duarte, Inês Bernardino, Federica Pelle, Stefano Francione, Francisco Sales, and Miguel Castelo-Branco. 2022. Intracranial recordings in humans reveal specific hippocampal spectral and dorsal vs. ventral connectivity signatures during visual, attention and memory tasks. *Scientific Reports* 12 (2022).
- [17] Moo Sung Chae, Zhi Yang, Mehmet R Yuce, Linh Hoang, and Wentai Liu. 2009. A 128-channel 6 mW wireless neural recording IC with spike feature extraction and UWB transmitter. *IEEE transactions on neural systems and rehabilitation engineering* 17 (2009).

- [18] Li-Pin Chang, Tei-Wei Kuo, and Shi-Wu Lo. 2004. Real-time garbage collection for flash-memory storage systems of real-time embedded systems. *ACM Trans. Embed. Comput. Syst.* 3 (2004).
- [19] John K Chapin, Karen A Moxon, Ronald S Markowitz, and Miguel AL Nicolelis. 1999. Real-time control of a robot arm using simultaneously recorded neurons in the motor cortex. *Nature neuroscience* 2 (1999).
- [20] Poonam Chaudhary and Rashmi Agrawal. 2018. Emerging threats to security and privacy in brain computer interface. *International Journal of Advanced Studies of Scientific Research* 3, 12 (2018).
- [21] An Chen. 2016. A review of emerging non-volatile memory (NVM) technologies and applications. *Solid-State Electronics* 125 (2016).
- [22] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. <https://arxiv.org/abs/1604.06174>.
- [23] Xubin Chen, Yin Li, Jingpeng Hao, Hyunsuk Shin, Michael Suh, and Tong Zhang. 2019. Simultaneously reducing cost and improving performance of NVM-based block devices via transparent data compression. In *Proceedings of the International Symposium on Memory Systems*.
- [24] Yingping Chen, Bernardo Tacca, Yunzhu Chen, Dwaipayan Biswas, Georges Gielen, Francky Cathoor, Marian Verhelst, and Carolina Mora Lopez. 2023. An Online-Spike-Sorting IC Using Unsupervised Geometry-Aware OSort Clustering for Efficient Embedded Neural-Signal Processing. *IEEE Journal of Solid-State Circuits* (2023).
- [25] Sen Cheng and Loren M Frank. 2008. New experiences enhance coordinated neural activity in the hippocampus. *Neuron* 57 (2008).
- [26] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. <https://arxiv.org/abs/1406.1078>.
- [27] Adelson Chua, Michael I. Jordan, and Rikky Muller. 2022. SOUL: An Energy-Efficient Unsupervised Online Learning Seizure Detection Classifier. *IEEE Journal of Solid-State Circuits* 57 (2022).
- [28] Davide Ciliberti, Frédéric Michon, and Fabian Kloosterman. 2018. Real-time classification of experience-related ensemble spiking patterns for closed-loop applications. *Elife* 7 (2018).
- [29] Dhananjay Das, Dulcardo Arteaga, Nisha Talagala, Torben Mathiasen, and Jan Lindström. 2014. NVM Compression—Hybrid Flash-Aware Application Level Compression. In *2nd Workshop on Interactions of NVM/Flash with Operating Systems and Workloads (INFLOW 14)*.
- [30] Alan D Degenhart, William E Bishop, Emily R Oby, Elizabeth C Tyler-Kabara, Steven M Chase, Aaron P Batista, and Byron M Yu. 2020. Stabilization of a brain-computer interface via the alignment of low-dimensional spaces of neural activity. *Nature biomedical engineering* 4, 7 (2020), 672–685.
- [31] Maryam Dehghani, Ali Mobaeni, and Reza Boostani. 2021. A deep neural network-based transfer learning to enhance the performance and learning speed of BCI systems. *Brain-Computer Interfaces* 8 (2021).
- [32] Julie Dethier, Vikash Gilja, Paul Nuyujukian, Shaqui A Ellassaad, Krishna V Shenoy, and Kwabena Boahen. 2011. Spiking neural network decoder for brain-machine interfaces. In *Proc. 5th International IEEE/EMBS Conference on Neural Engineering (EMBS)*.
- [33] Johannes P.F. D'Haeyer. 1989. Gaussian filtering of images: A regularization approach. *Signal Processing* 18 (1989).
- [34] Barundeb Dutta, Alexandru Andrei, TD Harris, CM Lopez, J O'Callahan, Jan Putzeys, BC Raducanu, Simone Severi, SD Stavisky, EM Trautmann, M. Welkenhuyzen, and K.V. Shenoy. 2019. The Neuropixels probe: A CMOS based integrated microsystems platform for neuroscience and brain-computer interfaces. In *Proc. 2019 IEEE International Electron Devices Meeting (IEDM)*.
- [35] Chaoifei Fan, Nick Hahn, Foram Kamdar, Donald Avansino, Guy Wilson, Leigh Hochberg, Krishna V Shenoy, Jaimie Henderson, and Francis Willett. 2023. Plug-and-play stability for intracortical brain-computer interfaces: a one-year demonstration of seamless brain-to-text communication. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [36] Luís Meruje Ferreira, Fábio Coelho, and José Orlando Pereira. 2023. Towards MRAM Byte-Addressable Persistent Memory in Edge Database Systems.. In *VLDB Workshops*.
- [37] Robert Fisher, Vicenta Salanova, Thomas Witt, Robert Worth, Thomas Henry, Robert Gross, Kalarickal Oommen, Ivan Osorio, Jules Nazzaro, Douglas Labar, Michael Kaplitt, Michael Sperling, Evan Sandok, John Neal, Adrian Handforth, John Stern, Antonio DeSalles, Steve Chung, Andrew Shetter, Donna Bergen, Roy Bakay, Jaimie Henderson, Jacqueline French, Gordon Baltuch, William Rosenfeld, Andrew Youkalis, William Marks, Paul Garcia, Nicolas Barbaro, Nathan Fountain, Carl Bazil, Robert Goodman, Guy McKhann, K. Babu Krishnamurthy, Steven Papavassiliou, Charles Epstein, John Pollard, Lisa Tonder, Joan Grebin, Robert Coffey, Nina Graves, and the SANTE Study Group. 2010. Electrical stimulation of the anterior nucleus of thalamus for treatment of refractory epilepsy. *Epilepsia* 51 (2010).
- [38] Fred H Gage. 2002. Neurogenesis in the adult brain. *Journal of neuroscience* 22 (2002).
- [39] Samuel Garcia, Charlie Windolf, Julien Boussard, Benjamin Dichter, Alessio P Buccino, and Pierre Yger. 2024. A Modular Implementation to Handle and Benchmark Drift Correction for High-Density Extracellular Recordings. *Eneuro* 11 (2024).
- [40] Pierre Gergondet, Abderrahmane Kheddar, Christoph Hintermüller, Christoph Guger, and Mel Slater. 2013. Multitask humanoid control with a brain-computer interface: user experiment with HRP-2. In *Proc. 13th International Symposium on Experimental Robotics (ISER)*.
- [41] Sarah Gibson, Jack W. Judy, and Dejan Marković. 2012. Spike Sorting: The First Step in Decoding the Brain: The first step in decoding the brain. *IEEE Signal Processing Magazine* 29 (2012).
- [42] Zhejun Guo, Fang Wang, Longchun Wang, Kejun Tu, Chunpeng Jiang, Ye Xi, Wen Hong, Qingda Xu, Xiaolin Wang, Bin Yang, Bomin Sun, Zude Lin, and Jingquan Liu. 2022. A flexible neural implant with ultrathin substrate for low-invasive brain-computer interface applications. *Microsystems & Nanoengineering* 8 (2022).
- [43] Matthew R. Guthaus, James E. Stine, Samira Ateei, Brian Chen, Bin Wu, and Mehedi Sarwar. 2016. OpenRAM: An open-source memory compiler. In *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.
- [44] Jiawei Han, Hongjie Jiang, and Junming Zhu. 2020. Neurorestoration: Advances in human brain-computer interface using microelectrode arrays. *Journal of Neurorestoratology* 8 (2020).
- [45] Christian Herff, Lorenz Diener, Miguel Angrick, Emily Mugler, Matthew C Tate, Matthew A Goldrick, Dean J Krusienski, Marc W Slutzky, and Tanja Schultz. 2019. Generating natural, intelligible speech from brain activity in motor, premotor, and inferior frontal cortices. *Frontiers in neuroscience* 13 (2019).
- [46] Xiao-Yu Hu, Evangelos Eleftheriou, Robert Haas, Ilias Iliadis, and Roman Pletka. 2009. Write amplification analysis in flash-based solid state drives. In *Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference*.
- [47] Zichen Hu, Zhining Zhou, and Hongming Lyu. 2024. A Power-and-Area-Efficient Channel-Interleaved Neural Signal Processor for Wireless Brain-Computer Interfaces with Unsupervised Spike Sorting. *IEEE Transactions on Biomedical Circuits and Systems* (2024).
- [48] Binbin Huang, Hailian Zhao, Lingbin Wang, Wenzhuo Qian, Yuyu Yin, and Shiguang Deng. 2024. Decentralized Proactive Model Offloading and Resource Allocation for Split and Federated Learning. *IEEE Internet of Things Journal* (2024).
- [49] Beata Jarosiewicz, Anish A. Sarma, Daniel Bacher, Nicolas Y. Masse, John D. Simeral, Brittany Sorice, Erin M. Oakley, Christine Blabe, Chethan Pandarinath, Vikash Gilja, Sydney S. Cash, Emad N. Eskandar, Gerhard Friehs, Jaimie M. Henderson, Krishna V. Shenoy, John P. Donoghue, and Leigh R. Hochberg. 2015. Virtual typing by people with tetraplegia using a self-calibrating intracortical brain-computer interface. *Science Translational Medicine* 7 (2015).
- [50] Hui Jiang, Min Liu, Sheng Sun, Yuwei Wang, and Xiaobing Guo. 2022. FedSyl: Computation-efficient federated synergy learning on heterogeneous iot devices. In *2022 IEEE/ACM 30th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.
- [51] Song Jiang, Xiaoning Ding, Feng Chen, Enhua Tan, and Xiaodong Zhang. 2005. DULO: an effective buffer cache management scheme to exploit both temporal and spatial locality. In *Proceedings of the 4th conference on USENIX Conference on File and Storage Technologies*.
- [52] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. 2018. SimpleSSD: Modeling Solid State Drives for Holistic System Simulation. *IEEE Computer Architecture Letters* 17 (2018).
- [53] Ioannis Karageorgos, Karthik Sriram, Ján Veselý, Michael Wu, Marc Powell, David Borton, Rajit Manohar, and Abhishek Bhattacharjee. 2020. Hardware-software co-design for brain-computer interfaces. In *Proc. 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.
- [54] Kaan Karas, Luca Pozzi, Alessandra Pedrocchi, Francesco Braghin, and Loris Roveda. 2023. Brain-computer interface for robot control with eye artifacts for assistive applications. *Scientific Reports* 13 (2023).
- [55] Saeed Kargar and Faisal Nawab. 2023. Challenges and future directions for energy, latency, and lifetime improvements in NVMs. *Distributed and Parallel Databases* 41, 3 (2023).
- [56] Vaibhav Karkare, Sarah Gibson, and Dejan Marković. 2013. A 75-μw, 16-channel neural spike-sorting processor with unsupervised clustering. *IEEE Journal of Solid-State Circuits* 48 (2013).
- [57] Mattias P Karlsson and Loren M Frank. 2009. Awake replay of remote experiences in the hippocampus. *Nature neuroscience* 12 (2009).
- [58] Brianna M Karpowicz, Bareesh Bhaduri, Samuel R Nason-Tomaszewski, Brandon G Jacques, Yahia H Ali, Robert D Flint, Payton H Bechefskey, Leigh R Hochberg, Nicholas AuYong, Marc W Slutzky, and Chethan Pandarinath. 2024. Reducing power requirements for high-accuracy decoding in iBCIs. *Journal of Neural Engineering* 21 (2024).
- [59] Hossein Kassiri, Sana Tonekaboni, M Tariqus Salam, Nima Soltani, Karim Abdelhalim, Jose Luis Perez Velazquez, and Roman Genov. 2017. Closed-loop neurostimulators: A survey and a seizure-predicting design example for intractable

- epilepsy treatment. *IEEE transactions on biomedical circuits and systems* 11 (2017).
- [60] Jill L Kays, Robin A Hurley, and Katherine H Taber. 2012. The dynamic brain: neuroplasticity and mental health. *The Journal of neuropsychiatry and clinical neurosciences* 24 (2012).
- [61] S. Keerthi, Vikas Sindhwani, and Olivier Chapelle. 2006. An Efficient Method for Gradient-Based Adaptation of Hyperparameters in SVM Models. In *Advances in Neural Information Processing Systems*.
- [62] Chris Dongjoo Kim, Jinseo Jeong, and Gunhee Kim. 2020. Imbalanced continual learning with partitioning reservoir sampling. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIII 16*. Springer, 411–428.
- [63] Hyojun Kim and Seongjun Ahn. 2008. BPLRU: A Buffer Management Scheme for Improving Random Writes in Flash Storage. In *FAST*.
- [64] Sohee Kim, Prashant Tathireddy, Richard A Normann, and Florian Solzbacher. 2007. Thermal impact of an active 3-D microelectrode array implanted in the brain. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 15 (2007).
- [65] Eran Klein. 2016. Informed consent in implantable BCI research: identifying risks and exploring meaning. *Science and Engineering Ethics* 22 (2016).
- [66] Simeon Knieling, Kousik S. Sridharan, Paolo Belardinelli, Georgios Naros, Daniel Weiss, Florian Mormann, and Alireza Gharabaghi. 2016. An Unsupervised Online Spike-Sorting Framework. *International Journal of Neural Systems* 26 (2016).
- [67] Hunjun Lee, Yeongwoo Jang, Daye Jung, Seunghyun Song, and Jangwoo Kim. 2024. Rearchitecting a Neuromorphic Processor for Spike-Driven Brain-Computer Interfacing. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1073–1089.
- [68] Eric C Leuthardt, Daniel W Moran, and Tim R Mullen. 2021. Defining surgical terminology and risk for brain computer interface technologies. *Frontiers in Neuroscience* 15 (2021).
- [69] QianQian Li, Ding Ding, and Mauro Conti. 2015. Brain-computer interface applications: Security and privacy challenges. In *2015 IEEE conference on communications and network security (CNS)*.
- [70] Jianwei Liao, Fengxiang Zhang, Li Li, and Guoqiang Xiao. 2015. Adaptive Wear-Leveling in Flash-Based Memory. *IEEE Computer Architecture Letters* (2015).
- [71] Michael D. Linderman, Vikash Gilja, Gopal Santhanam, Afsheen Afshar, Stephen Ryu, Teresa H. Meng, and Krishna V. Shenoy. 2006. Neural Recording Stability of Chronic Electrode Arrays in Freely Behaving Primates. In *2006 International Conference of the IEEE Engineering in Medicine and Biology Society*.
- [72] David Lopez-Paz and Marc'Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. *Advances in neural information processing systems* 30 (2017).
- [73] Kenway Louie and Matthew A Wilson. 2001. Temporally structured replay of awake hippocampal ensemble activity during rapid eye movement sleep. *Neuron* 29 (2001).
- [74] Shuxia Lu and Zhao Jin. 2017. International Journal of Recent Engineering Science (IJRES). *International Journal of Recent Engineering Science (IJRES)* 4 (2017).
- [75] Youyou Lu, Jiwei Shu, and Weimin Zheng. 2013. Extending the Lifetime of Flash-based Storage through Reducing Write Amplification from File Systems. In *11th USENIX Conference on File and Storage Technologies (FAST 13)*.
- [76] Xuan Ma, Fabio Rizzoglio, Kevin L Bodkin, Eric Perreault, Lee E Miller, and Ann Kennedy. 2023. Using adversarial networks to extend brain computer interface decoding accuracy over time. *Elife* 12 (2023).
- [77] Jeremy Magland, James J Jun, Elizabeth Lovero, Alexander J Morley, Cole Lincoln Hurwitz, Alessio Paolo Buccino, Samuel Garcia, and Alex H Barnett. 2020. SpikeForest, reproducible web-facing ground-truth validation of automated neural spike sorters. *eLife* 9 (2020).
- [78] Eve Marder and Jean-Marc Goaillard. 2006. Variability, compensation and homeostasis in neuron and network function. *Nature Reviews Neuroscience* 7 (2006).
- [79] Lan Mei, Thorir Mar Ingolfsson, Cristian Cioflan, Victor Kartsch, Andrea Cossetti, Xiaying Wang, and Luca Benini. 2024. An Ultra-Low Power Wearable BMI System with Continual Learning Capabilities. *IEEE Transactions on Biomedical Circuits and Systems* (2024).
- [80] Inc. Micron Technology. [n.d.]. MT29F8G08ADAFWP-AAT-F. <https://www.micron.com/products/storage/nand-flash/slc-nand/part-catalog/part-detail/mt29f8g08adafawp-aat-f>.
- [81] Christian Monzio Compagnoni, Akira Goda, Alessandro S. Spinelli, Peter Feeley, Andrea L. Lacaita, and Angelo Visconti. 2017. Reviewing the Evolution of the NAND Flash Technology. *Proc. IEEE* 105 (2017).
- [82] Elon Musk and Neuralink. 2019. An integrated brain-machine interface platform with thousands of channels. *Journal of medical Internet research* 21 (2019).
- [83] Neuropace. 2021. RNS System Patient Manual.
- [84] Soroush Niketeghad and Nader Pouratian. 2019. Brain machine interfaces for vision restoration: the current state of cortical visual prosthetics. *Neurotherapeutics* 16 (2019).
- [85] Gerard O'Leary, David M Groppe, Taufik A Valiante, Naveen Verma, and Roman Genov. 2018. NURIP: Neural interface processor for brain-state classification and programmable-waveform neurostimulation. *IEEE Journal of Solid-State Circuits* 53 (2018).
- [86] Marius Pachitariu, Shashwat Sridhar, Jacob Pennington, and Carsen Stringer. 2024. Spike sorting with Kilosort4. *Nature Methods* 21 (2024).
- [87] Yangyang Pan, Guiqiang Dong, and Tong Zhang. 2013. Error Rate-Based Wear-Leveling for nand Flash Memory at Highly Scaled Technology Nodes. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2013).
- [88] Seon-yeong Park, Dawoon Jung, Jeong-uk Kang, Jin-soo Kim, and Joonwon Lee. 2006. CFLRU: a replacement algorithm for flash memory. In *Proceedings of the 2006 international conference on Compilers, architecture and synthesis for embedded systems*.
- [89] Youngjo Park and Jin-soo Kim. 2011. zFTL: power-efficient data compression support for NAND flash-based consumer electronics devices. *IEEE Transactions on Consumer Electronics* (2011).
- [90] Yun S Park, G Rees Cosgrove, Joseph R Madsen, Emad N Eskandar, Leigh R Hochberg, Sydney S Cash, and Wilson Truccolo. 2019. Early detection of human epileptic seizures based on intracortical microelectrode array signals. *IEEE Transactions on Biomedical Engineering* 67 (2019).
- [91] Felix Pei, Joel Ye, David M. Zoltowski, Anqi Wu, Raaed H. Chowdhury, Hansem Sohn, Joseph E. O'Doherty, Krishna V. Shenoy, Matthew T. Kaufman, Mark Churchland, Mehrdad Jazayeri, Lee E. Miller, Jonathan Pillow, Il Memming Park, Eva L. Dyer, and Chethan Pandarinath. 2021. Neural Latents Benchmark '21: Evaluating latent variable models of neural population activity. In *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks 1 (NeurIPS Datasets and Benchmarks)*.
- [92] Ivy B Peng, Maya B Gokhale, and Eric W Green. 2019. System evaluation of the intel optane byte-addressable nvm. In *Proceedings of the International Symposium on Memory Systems*. 304–315.
- [93] Raghavendra Pradyumna Pothukuchi, Karthik Sriram, Michał Gerasimiuk, Muhammed Ugur, Rajit Manohar, Anurag Khandelwal, and Abhishek Bhat-tacharjee. 2024. Distributed Brain-Computer Interfacing With a Networked Multiaccelerator Architecture. *IEEE Micro* 44 (2024).
- [94] Bogdan C Raducanu, Refet F Yazicioglu, Carolina M Lopez, Marco Ballini, Jan Putzeys, Shiwei Wang, Alexandru Andrei, Veronique Rochus, Marleen Welkenhuysen, Nick Van Helleputte, Silke Musa, Robert Puers, Fabian Kloosterman, Chris van Hoof, Richárd Fiáth, István Ulbert, and Srinjoy Mitra. 2017. Time multiplexed active neural probe with 1356 parallel recording sites. *Sensors* 17 (2017).
- [95] Mendel Rosenblum and John K. Ousterhout. 1992. The design and implementation of a log-structured file system. *ACM Trans. Comput. Syst.* 10 (1992).
- [96] Cyrille Rossant, Shabnam N Kadir, Dan FM Goodman, John Schulman, Maximilian LD Hunter, Aman B Saleem, Andres Grosmark, Mariano Belluscio, George H Denfield, Alexander S Ecker, Andreas S Tolias, Samuel Solomon, György Buzsáki, Matteo Carandini, and Kenneth D Harris. 2016. Spike sorting for large, dense electrode arrays. *Nature neuroscience* 19 (2016).
- [97] Sebastian Ruder. 2016. An overview of gradient descent optimization algorithms. <https://arxiv.org/abs/1609.04747>.
- [98] Ueli Rutishauser, Erin M Schuman, and Adam N Mamelak. 2006. Online detection and sorting of extracellularly recorded action potentials in human medial temporal lobe recordings, in vivo. *Journal of neuroscience methods* 154 (2006).
- [99] Eric Samikwa, Antonio Di Maio, and Torsten Braun. 2022. Ares: Adaptive resource-aware split learning for internet of things. *Computer Networks* 218 (2022), 109380.
- [100] Abdul Satti, Damien Coyle, and Girijesh Prasad. 2009. Continuous EEG classification for a self-paced BCI. In *2009 4th International IEEE/EMBS Conference on Neural Engineering*.
- [101] Claudia Serrano-Amenos, Payam Heydari, Charles Y Liu, An H Do, and Zoran Nenadic. 2023. Power budget of a skull unit in a fully-implantable brain-computer interface: Bio-heat model. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 31 (2023).
- [102] Eugene Sha, Andy Liu, Kareem Ibrahim, Mostafa Mahmoud, Christina Gian-noula, Ameer Abdelhadi, and Andreas Moshovos. 2024. Marple: Scalable Spike Sorting for Untethered Brain-Machine Interfacing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.
- [103] Xin Shi, Fei Wu, Shunzhuo Wang, Changsheng Xie, and Zhonghai Lu. 2018. Program error rate-based wear leveling for NAND flash memory. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*.
- [104] Uisub Shin, Cong Ding, Bingzhao Zhu, Yashwanth Vyza, Alix Trouillet, Emilie CM Revol, Stéphanie P Lacour, and Mahsa Shoaran. 2022. NeuralTree: A 256-channel 0.227-μJ/class versatile neural activity classification and closed-loop neuromodulation SoC. *IEEE Journal of Solid-State Circuits* (2022).
- [105] Jennifer Shum and Daniel Friedman. 2021. Commercially available seizure detection devices: A systematic review. *Journal of the Neurological Sciences* 428 (2021).

- [106] Nicholas D Skomrock, Michael A Schwemmer, Jordyn E Ting, Hemang R Trivedi, Gaurav Sharma, Marcia A Bockbrader, and David A Friedenberg. 2018. A Characterization of Brain-Computer Interface Performance Trade-Offs Using Support Vector Machines and Deep Neural Networks to Decode Movement Intent. *Frontiers in neuroscience* 12 (2018).
- [107] Ethan Sorrell, Michael E Rule, and Timothy O'Leary. 2021. Brain-machine interfaces: closed-loop control in an adaptive system. *Annual Review of Control, Robotics, and Autonomous Systems* (2021).
- [108] Karthik Sriram, Raghavendra Pradyumna Pothukuchi, Michal Gerasimiuk, Muhammed Ugur, Oliver Ye, Rajit Manohar, Anurag Khandelwal, and Abhishek Bhattacharjee. 2023. SCALO: An Accelerator-Rich Distributed System for Scalable Brain-Computer Interfacing. In *Proc. 50th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.
- [109] Scott Stanslaski, Jeffrey Herron, Tom Chouinard, Duane Bourget, Ben Isaacson, Václav Kremen, Enrico Opri, William Drew, Benjamin H. Brinkmann, Aysegul Gunduz, Tom Adamski, Gregory A. Worrell, and Timothy Denison. 2018. A chronically implantable neural coprocessor for investigating the treatment of neurological disorders. *IEEE transactions on biomedical circuits and systems* 12 (2018).
- [110] James E. Stine, Ivan Castellanos, Michael Wood, Fejj Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *Proc. 2007 IEEE International Conference on Microelectronic Systems Education (MSE)*.
- [111] Felice T Sun and Martha J Morrell. 2014. The RNS System: responsive cortical stimulation for the treatment of refractory partial epilepsy. *Expert review of medical devices* 11 (2014).
- [112] Akihiko Tsukahara, Yuki Anzai, Keita Tanaka, and Yoshinori Uchikawa. 2021. A design of EEGNet-based inference processor for pattern recognition of EEG using FPGA. *Electronics and Communications in Japan* (2021).
- [113] Muhammed Ugur, Raghavendra Pradyumna Pothukuchi, and Abhishek Bhattacharjee. 2024. Leveraging NVMs for Neural Interface Coverage. The 1st Workshop on Emerging Computer Systems Challenges and Applications in Biomedicine (BioSys).
- [114] Muhammed Ugur, Raghavendra Pradyumna Pothukuchi, and Abhishek Bhattacharjee. 2024. Swapping-Centric Neural Recording Systems. In *The 15th Annual Non-Volatile Memories Workshop*.
- [115] Muhammed Ugur, Raghavendra Pradyumna Pothukuchi, and Abhishek Bhattacharjee. 2024. Towards Forever Access for Implanted Brain-Computer Interfaces. <https://arxiv.org/abs/2409.17496>.
- [116] Daniel Valencia and Amirhossein Alimohammad. 2019. An efficient hardware architecture for template matching-based spike sorting. *IEEE transactions on biomedical circuits and systems* (2019).
- [117] Mariska J. Vansteensel, Elmar G.M. Pels, Martin G. Bleichner, Mariana P. Branco, Timothy Denison, Zachary V. Freudenburg, Peter Gosselaar, Sacha Leinders, Thomas H. Ottens, Max A. Van Den Boom, Peter C. Van Rijen, Erik J. Aarnoutse, and Nick F. Ramsey. 2016. Fully Implanted Brain-Computer Interface in a Locked-In Patient with ALS. *New England Journal of Medicine* 375 (2016).
- [118] Taige Wang, Mahsa Shooran, and Azita Emami. 2018. Towards Adaptive Deep Brain Stimulation in Parkinson's Disease: Lfp-Based Feature Analysis and Classification. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- [119] Johan Wessberg and Miguel AL Nicolelis. 2004. Optimizing a Linear Algorithm for Real-Time Robotic Control using Chronic Cortical Ensemble Recordings in Monkeys. *Journal of Cognitive Neuroscience* 16 (2004).
- [120] Francis R Willett, Donald T Avansino, Leigh R Hochberg, Jaimie M Henderson, and Krishna V Shenoy. 2021. High-performance brain-to-text communication via handwriting. *Nature* 593 (2021).
- [121] Charlie Windolf, Han Yu, Angelique C. Paulk, Domokos Meszéna, William Muñoz, Julien Boussard, Richard Hardstone, Irene Caprara, Mohsen Jamali, Yoav Kfir, Duo Xu, Jason E. Chung, Kristin K. Sellers, Zhiwen Ye, Jordan Shaker, Anna Lebedeva, Manu Raghavan, Eric Trautmann, Max Melin, João Couto, Samuel Garcia, Brian Coughlin, Csaba Horváth, Richárd Fiáth, István Ulbert, J. Anthony Movshon, Michael N. Shadlen, Mark M. Churchland, Anne K. Churchland, Nicholas A. Steinmetz, Edward F. Chang, Jeffrey S. Schweitzer, Ziv M. Williams, Sydney S. Cash, Liam Paninski, and Erdem Varol. 2023. DREDge: robust motion correction for high-density extracellular recordings across species. *bioRxiv* (2023).
- [122] Rolf Winkler and Alexa Corse. 2024. Neuralink's First Brain-Chip Implant in a Human Appeared Flawless. There Was a Problem. *The Wall Street Journal*. <https://www.wsj.com/tech/neuralinks-first-brain-chip-implant-in-a-human-appeared-flawless-there-was-a-problem-637f1adc?st=1rf66s4cobkefpa>
- [123] Adrien Wohrer, Mark D Humphries, and Christian K Machens. 2013. Population-wide distributions of neural activity during perceptual decision-making. *Progress in neurobiology* 103 (2013).
- [124] Patrick D Wolf and WM Reichert. 2008. Thermal considerations for the design of an implanted cortical brain-machine interface (BMI). *Indwelling Neural Implants: Strategies for Contending with the In Vivo Environment* (2008).
- [125] Di Wu, Jingjie Li, Zhewen Pan, Younghyun Kim, and Joshua San Miguel. 2022. uBrain: A unary brain computer interface. In *Proc. 49th Annual International Symposium on Computer Architecture (ISCA)*.
- [126] Kun Xia, Włodzisław Duch, Yu Sun, Kedi Xu, Weili Fang, Hanbin Luo, Yi Zhang, Dong Sang, Xiaodong Xu, and Fei-Yue Wang. 2022. Privacy-preserving brain-computer interfaces: A systematic review. *IEEE Transactions on Computational Social Systems* (2022).
- [127] Dongkuan Xu and Yingjie Tian. 2015. A Comprehensive Survey of Clustering Algorithms. *Annals of Data Science* 2 (2015).
- [128] Ming-Chang Yang, Yu-Ming Chang, Che-Wei Tsao, Po-Chun Huang, Yuan-Hao Chang, and Tei-Wei Kuo. 2014. Garbage collection and wear leveling for flash memory: Past and future. In *2014 International Conference on Smart Computing*.
- [129] Yikai Yang, Nhan Duy Truong, Jason K Eshraghian, Armin Nikpour, and Omid Kavehei. 2022. Weak self-supervised learning for seizure forecasting: a feasibility study. *Royal Society Open Science* 9 (2022).
- [130] Do-Yeon Yoon, Sonal Pinto, SungWon Chung, Paul Merolla, Thong-Wei Koh, and Dongjin Seo. 2021. A 1024-channel simultaneous recording neural SoC with stimulation and real-time spike detection. In *Proc. 2021 Symposium on VLSI Circuits (VLSIC)*.
- [131] Hamed Zaer, Ashlesha Deshmukh, Dariusz Orlowski, Wei Fan, Pierre-Hugues Prouvot, Andreas Norgaard Glud, Morten Bjørn Jensen, Esben Schjødtt Worm, Slávka Lukacova, Trine Werenberg Mikkelsen, Lise Moberg Fitting, John R. Adler, M. Bret Schneider, Martin Snejbjerg Jensen, Quanhai Fu, Vinson Go, James Morizio, Jens Christian Hedemann Sørensen, and Albrecht Stroh. 2021. An Intracortical Implantable Brain-Computer Interface for Telemetric Real-Time Recording and Manipulation of Neuronal Circuits for Closed-Loop Intervention. *Frontiers in Human Neuroscience* 15 (2021).
- [132] Majid Zamani, Dai Jiang, and Andreas Demosthenous. 2017. A highly accurate spike sorting processor with reconfigurable embedded frames for unsupervised and adaptive analysis of neural signals. In *ESSCIRC 2017 - 43rd IEEE European Solid State Circuits Conference*.
- [133] Mingming Zhang, Michael A Schwemmer, Jordyn E Ting, Connor E Majstorovic, David A Friedenberg, Marcia A Bockbrader, W Jerry Mysiw, Ali R Rezai, Nicholas V Annetta, Chad E Bouton, Herbert S Bresler, and Gaurav Sharma. 2018. Extracting wavelet based neural features from human intracortical recordings for neuroprosthetics applications. *Bioelectronic Medicine* (2018).
- [134] Miaolin Zhang, Lian Zhang, Jeong Hoan Park, Chne-Wuen Tsai, Kian Ann Ng, Longyang Lin, Yilong Dong, Jiamin Li, Tao Tang, Han Wu, Liuhao Wu, and Jerald Yoo. 2021. A One-Shot Learning, Online-Tuning, Closed-Loop Epilepsy Management SoC with 0.97 μ J/Classification and 97.8% Vector-Based Sensitivity. In *Proc. 2021 Symposium on VLSI Circuits (VLSIC)*.
- [135] Tim Zhang, Mostafa Rahimi Azghadi, Corey Lammie, Amirali Amirsoleimani, and Roman Genov. 2023. Spike sorting algorithms and their efficient hardware implementation: a comprehensive survey. *Journal of Neural Engineering* (2023).
- [136] Yujuan Zhao, Lin Tang, Robert Rennaker, Chris Hutchens, and Tamer S Ibrahim. 2013. Studies in RF power communication, SAR, and temperature elevation in wireless implantable neural interfaces. *PloS one* 8 (2013).
- [137] J. Ziv and A. Lempel. 1977. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory* 23 (1977).
- [138] J. Ziv and A. Lempel. 1978. Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory* (1978).